

A

The ANSI Runtime Library

Until recently, each compiler manufacturer delivered its own unique library of runtime routines and there was little effort toward standardization. This state of affairs wreaked havoc on programmers trying to write portable code. There was no guarantee that a program using library functions from one compiler manufacturer would still run when ported to another machine. With ratification of the ANSI Standard, this situation should improve.

The task of deriving a single set of routines, however, was no simple matter. The ANSI Committee was faced with literally dozens of existing functions, many with the same names but with different effects. One of the committee's main goals was to break as little existing code as possible. The result was a somewhat larger and less consistent library than anyone really desired. Nevertheless, once you learn some of its subtleties and quirks, the ANSI C library becomes a remarkably powerful tool.

The main source for the routines in the ANSI library is the UNIX operating system. Many of the functions supported by ANSI are exactly the same as those supported by System V and BSD4.2, and others are very similar. As of this writing, AT&T, in conjunction with the UNIX Standard IEEE Committee, appears committed to bringing its library into accord with the ANSI Standard.

A.1 Function Names

All library function names are reserved as external identifiers. Therefore, you cannot define a name that matches a library name, even if you are defining a function that performs the same operation. Likewise, all identifiers beginning with an underscore are reserved for behind-the-scenes macros. Appendix D provides a complete list of reserved names.

A.2 Header Files

In addition to the functions themselves, the C runtime library comes with a set of include files called *header files*. Every function is associated with one or more header files that must be included wherever the function is invoked. These files contain the declarations for any related functions, macros, or data types needed to execute a set of library functions. Table A-1 lists the standard header files. Note that some of these header files may not be available with your compiler if it is not ANSI-conforming.

The ANSI Standard guarantees that the header files are *idempotent*. This means that multiple inclusions of the same header file will not have adverse effects and you can include header files in any order.

A.3 Synopses

The form of each function and the header file it requires are provided in a format called a *synopsis*. The synopsis format is taken from UNIX documentation and has the following form:

```
#include header_file  
function_prototype
```

For example, the following is the synopsis for the function *gets()*, which reads a string from standard input:

```
#include <stdio.h>  
char *gets (char *s);
```

This tells you that you must include the header file *<stdio.h>* and that *gets()* takes one argument that is a pointer to a *char* and returns a pointer to a *char*. To use *gets()* in a program, you would include the line

```
#include <stdio.h>
```

in your source file. You do not need to declare *gets()* because any necessary declarations are performed in the header file. The angle brackets enclosing the filename inform the compiler to search for the header file in a system-defined location. As long as your C compiler is installed as directed

by the vendor, you should not need to worry about the real location of the header files.

Table A-1: Header Files for the Runtime Library.

Header File	Associated Functions
<code><assert.h></code>	Diagnostic functions (currently just the <i>assert()</i> macro).
<code><ctype.h></code>	Character testing and mapping functions.
<code><errno.h></code>	Defines several macros related to error reporting: <i>EDOM</i> , <i>ERANGE</i> , and <i>errno</i> .
<code><float.h></code>	Defines a number of macros that describe the characteristics of floating-point objects in the environment (see Appendix B).
<code><limits.h></code>	Contains parameter values that describe the execution environment (see Appendix B).
<code><locale.h></code>	The <i>setlocale()</i> function, which enables you to set locale parameters.
<code><math.h></code>	Double-precision mathematics functions.
<code><setjmp.h></code>	The <i>setjmp()</i> and <i>longjmp()</i> functions, which enable you to bypass the normal function call and return discipline.
<code><signal.h></code>	Functions that handle signals.
<code><stdarg.h></code>	Functions and macros for implementing functions that accept a variable number of arguments.
<code><stddef.h></code>	This is a new header file that contains definitions of six macros: <i>ptrdiff_t</i> , <i>size_t</i> , <i>NULL</i> , <i>wchar_t</i> , <i>offsetof</i> , and <i>errno</i> . These macros are also defined in any header file declaring functions that use them.
<code><stdio.h></code>	I/O functions.
<code><stdlib.h></code>	General utility functions.
<code><string.h></code>	String manipulation functions.
<code><time.h></code>	Time manipulation functions.

It is also possible to declare a library function explicitly without referring to a header file. To declare *gets()*, for instance, you could write

```
extern char *gets( char * );
```

However, this can be a dangerous practice for a couple of reasons. First, the function you declare may refer to other functions or macros that are defined in the header file. If you don't include the header file, the function may not work. Second, the library function may have a macro implementation that runs faster than the real function. By declaring it as a function, you force the system to use the function version, which makes your program less efficient. For these reasons, we recommend that you always include whatever header files are indicated in the synopsis.

A.4 Functions vs. Macros

As described in Chapter 11, it is often possible to implement a function more efficiently as a macro. Most C compiler developers take advantage of this capability by implementing many of the library functions as macros. However, because this introduces some potential side-effect problems, the ANSI Standard enforces some restrictions to protect you. First, the ANSI Standard ensures that in any macro implementation, an argument is expanded once and only once. This avoids the pitfalls associated with side-effect operators in the argument expression (see the Bug Alert on page 311). Second, the ANSI Standard guarantees that there is an actual function for each library function listed. This enables you to take the address of any library function. (*getc()* and *putc()* are exceptions to both these rules.)

In many cases, there exist both a function and a macro that perform the same operation. By default, the macro gets executed because it is usually faster. If you want to execute the function instead, you can explicitly **#undef** the macro. The following examples illustrate this principle.

Example 1 – Using the function as defined in the header file (it may be a macro or it may be a function)

```
#include <stdio.h>
.
.
s = gets(s);
.
.
```

Example 2 – Forcing use of a function instead of a macro

```
#include <stdio.h>
#undef gets
.
.
s = gets(s);
.
.
```

A.5 Error Handling

Most library functions return a special value when an error occurs. The error value differs from routine to routine and is listed in the description of each individual function. The special macro *NULL* is often returned as an error value for functions that return pointers. It is an implementation-defined null pointer constant. *NULL* is defined in all header files that require it.

In some cases, in addition to returning an error value, a function also assigns a special error code to a global variable (or macro) called *errno*. *errno* is declared in the *<errno.h>* header file and has type *int*. For most implementations of C, each possible *errno* value is associated with an error message that you can output with the *perror()* function. You can also assign the error message to a string with the *strerror()* function.

A.6 Diagnostics

The C runtime library contains one header file and one macro for outputting diagnostics. In addition, there are several preprocessor symbols and commands that can be utilized to print diagnostic information (see Chapter 11). The header file is called *<assert.h>*. It defines a macro called *assert* and refers to another macro called *NDEBUG*. *NDEBUG*, however, is not defined by *<assert.h>*. In fact, if *NDEBUG* is defined when *<assert.h>* is included, then all subsequent calls to *assert()* will have no effect. Hence, *NDEBUG* provides a useful mechanism for turning off diagnostics, as illustrated in the following example.

```
#define NDEBUG
#include <assert.h>
/* calls to assert() will have no effect */
.
.
```

A.6.1 The *assert()* Function

```
#include <assert.h>
void assert( int expression );
```

According to the ANSI Standard, *assert()* must be implemented as a macro. The *assert()* macro tests the value of *expression*. If it is nonzero, no action is taken, and zero is returned. If *expression* equals zero, *assert()* writes information about the program's current status to *stderr* and then calls *abort()*. The diagnostic information contains the value of the expression, the current source file name, and the current line number. The latter two values are taken from the preprocessor symbols *__FILE__* and

`__LINE__`. The `assert()` macro is used most frequently to test the status of a function call. For instance,

```
/* If fopen() returns zero, send status
 * information to stderr, and abort.
 */
assert( fp = fopen( "file","r" ) );
```

Note, however, that `fopen()` will not even be invoked if `NDEBUG` is defined.

A.7 Character Handling

There are two groups of character-handling functions, which are usually implemented as macros. The first group, called *character-testing functions*, checks to see whether the argument is a member of a particular set of characters. The second group, called *case-mapping functions*, changes a letter from uppercase to lowercase, or vice versa. These functions are *not* ASCII-biased. They should work with any existing character code, including EBCDIC and European codes. Note, however, that there is some variation concerning how these functions operate, depending on what character set is being used. In these cases, a minimum operation is defined for programs operating in a *C locale* (see Section A.8 for more information about locales).

A.7.1 Character-Testing Functions

All of the character-testing functions have a similar format. They accept an `int` as the argument and return a nonzero value if the argument is a member of a specified set of characters. Otherwise they return zero. If the value of the argument cannot be represented in an **unsigned char**, the results are undefined. The generic synopsis is

```
#include <ctype.h>
int func_name( int c );
```

Table A-2 lists all of the character-testing functions and the set of characters for which they test membership.

Table A-2: Character Testing Functions.

<i>Function</i>	<i>Membership Set</i>
<i>isalnum()</i>	Alphabetic and digit characters (any character for which <i>isalpha()</i> or <i>isdigit()</i> is true).
<i>isalpha()</i>	Alphabetic characters (any character for which <i>isupper()</i> or <i>islower()</i> is true, or any implementation defined set of characters for which <i>iscntrl()</i> , <i>isdigit()</i> , <i>ispunct()</i> , and <i>isspace()</i> are false. In the C locale, <i>isalpha()</i> is true only if <i>islower()</i> or <i>isupper()</i> is true).
<i>iscntrl()</i>	Control characters.
<i>isdigit()</i>	Decimal digit characters.
<i>isgraph()</i>	All printable characters except space characters.
<i>islower()</i>	Lowercase letters or any implementation-defined subset of characters for which <i>iscntrl()</i> , <i>isdigit()</i> , <i>ispunct()</i> , and <i>isspace()</i> are false.
<i>isprint()</i>	All printing characters, including space.
<i>ispunct()</i>	All printing characters except a space and characters for which <i>isalnum()</i> is true.
<i>isspace()</i>	A space(' '), form feed ('\f'), newline ('\n'), carriage return ('\r'), horizontal tab ('\t'), or vertical tab ('\v').
<i>isupper()</i>	Uppercase letter or any implementation-defined subset of characters for which <i>iscntrl()</i> , <i>isdigit()</i> , <i>ispunct()</i> , and <i>isspace()</i> are false.
<i>isxdigit()</i>	Hexadecimal digits.

A.7.2 Character Case-Mapping Functions

There are two case-mapping functions, one that changes a letter from uppercase to lowercase and another that changes a letter from lowercase to uppercase. Both functions take an `int` argument and return an `int`. If the argument is not relevant (i.e., it is not a letter or it is already the case to which it is being converted), it is returned unchanged. The synopses for the two functions are

```
#include <ctype.h>
int tolower( int c );
```

and

```
#include <ctype.h>
int toupper( int c );
```

A.8 Setting Locale Parameters

Although ANSI is a U.S. organization, the C Standards Committee endeavored to make the C language as universal as possible. Among the problems it confronted were

- Different alphabets and hence different character sets.
- Different collating sequences in the character set (the numeric codes for alphabetic characters are not always ordered as they are in ASCII or EBCDIC).
- Different methods of representing decimal points (a period in the U.S., but a comma in many European countries).
- Different ways of displaying times and dates.

A large part of the problem was solved by putting locale-defined behavior into library functions rather than the language itself. For example, the *isalpha()* function can return different results depending on what character set is being used. This flexibility, however, raised another problem. If a library function has different interpretations based on locale, how can you force one particular interpretation? Also, how can you find out dynamically, while a program is running, which locale-specific behavior the program will exhibit?

The ANSI Committee solved both of these problems by inventing a function called *setlocale()*, which enables you to select a specific locale setting or to discover the current locale setting. By changing the locale setting, you can immediately change the action of all relevant functions. Note that these changes occur at runtime, not at compile time.

The header file associated with *setlocale()* is *<locale.h>*. This file contains definitions of at least six macros that enable you to select a particular part of the C language that you want to affect with the new locale setting:

<i>LC_ALL</i>	The entire language
<i>LC_COLLATE</i>	The <i>strcoll()</i> function
<i>LC_CTYPE</i>	All of the character-handling functions
<i>LC_MONETARY</i>	The monetary formatting information returned by the <i>localeconv()</i> function
<i>LC_NUMERIC</i>	The decimal point character for the formatted I/O and string conversion functions
<i>LC_TIME</i>	The <i>strftime()</i> function

Implementations may define additional macros that begin with *LC_*. In addition to the macros, the *locale.h* header file also defines a structure type named *lconv*. The elements of this structure are described in Section A.8.2.

A.8.1 The *setlocale()* Function

```
#include <locale.h>
char *setlocale( int category, const char *locale);
```

The *setlocale()* function sets or queries locale-specific behavior for the part of the C language specified by *category*. *category* should be one of the macros defined in *locale.h*.

If *locale* is a null pointer, the function is interpreted as a query. It returns a string that represents the current locale setting for the specified category. The only ANSI-defined locale is “C,” which represents the minimal environment for C translation. Implementations are free to define other locale strings.

You can find out what the implementation-defined locale is by entering a null string as the second parameter. In this case, *setlocale()* returns a pointer to the string associated with the specified category. The program's current locale is not changed.

If *locale* is not a null pointer, the function is interpreted as a request to change the locale setting for the specified category. If the request can be honored, *setlocale()* returns the *locale* argument. If the request cannot be honored, *setlocale()* returns a null pointer.

At program start-up, the equivalent of

```
setlocale( LC_ALL, "C" );
```

is executed.

A.8.2 The *localeconv()* Function

```
#include <locale.h>
struct lconv *localeconv( void );
```

The *localeconv()* function returns a pointer to a structure of type *lconv* that contains appropriate values for formatting numbers and monetary values under the current locale. The *lconv* structure contains the members shown in Table A-3. Any member with type *char ** (except *decimal_point*) can point to a null string, indicating that the locale does not define a format for this object. Likewise, any member with type *char* can have the value *CHAR_MAX* to indicate that there is no locale-defined value for that member.

Table A-3: *lconv* Structure.

<i>Structure Member</i>	<i>Meaning</i>
<code>char *decimal_point</code>	The decimal-point character used to format nonmonetary values.
<code>char *thousands_sep</code>	The character used to separate groups of digits (to the left of the decimal point) for nonmonetary values.
<code>char *grouping</code>	A string whose elements indicate the size of each group of digits in a nonmonetary value. The string can contain characters with the following values: <i>integer value</i> The number of digits that comprise the group. <i>0</i> Use the previous element to determine the number of digits to use for the current group. <i>CHAR_MAX</i> No further grouping is to be performed.
<code>char *int_curr_symbol</code>	The international currency symbol used by the current locale. The first three characters indicate the currency symbol as specified by ISO 4217. The fourth character defines the character used to separate the currency symbol from the monetary unit.
<code>char *currency_symbol</code>	The local currency symbol used by the current locale.
<code>char *mon_decimal_point</code>	The decimal point used to format monetary values.
<code>char *mon_thousands_sep</code>	The character used to separate groups of digits (to the left of the decimal point) for monetary values.
<code>char *mon_grouping</code>	A string whose elements indicate the size of each group of digits in a monetary value (see the description of grouping).
<code>char *negative_sign</code>	The string used to indicate a negative monetary value.

Table A-3: (continued)

<i>Structure Member</i>	<i>Meaning</i>
<code>char int_frac_digits</code>	The number of fractional digits (to the right of the decimal point) that are displayed in an internationally formatted monetary value.
<code>char frac_digits</code>	The number of fractional digits (to the right of the decimal point) that are displayed in a monetary value.
<code>char p_cs_precedes</code>	Set to 1 if the currency symbol precedes the monetary value when the value is positive, or to 0 if the symbol follows the monetary value.
<code>char p_sep_by_space</code>	Set to 1 if a space separates the currency symbol from a positive monetary value, or to 0 if there is no space.
<code>char n_cs_precedes</code>	Set to 1 if the currency symbol precedes the monetary value when the value is negative, or to 0 if the symbol follows the monetary value.
<code>char n_sep_by_space</code>	Set to 1 if a space separates the currency symbol from a negative monetary value, or to 0 if there is no space.
<code>char p_sign_posn</code>	Indicates where the positive sign is positioned for monetary values.
<code>char n_sign_posn</code>	Indicates where the negative sign is positioned for monetary values. <i>p_sign_posn</i> and <i>n_sign_posn</i> can have the following values:
	0 Parentheses surround the value and the currency symbol to indicate the sign.
	1 The sign symbol precedes the value and currency symbol.
	2 The sign symbol follows the value and currency symbol.
	3 The sign symbol immediately precedes the currency symbol.
	4 The sign symbol immediately follows the currency symbol.

A.9 Mathematics

All the math functions require inclusion of the header file `<math.h>`. This file contains a definition of the `HUGE_VAL` macro, which defines the value returned by all functions when the true result is too large to be represented.

There are two types of errors that can occur: *domain errors* and *range errors*. A *domain error* occurs when an input argument to the function is outside the legal domain for argument values. For example, it is a domain error to pass a negative number to the `sqrt()` function. In this case, the function returns an implementation-defined value and sets `errno` equal to the value of `EDOM`, which is an implementation-defined nonzero integer.

A *range error* occurs when the result of the function cannot be represented in a **double**. In this case, `errno` receives the value of the macro `ERANGE`, which again is an implementation-defined nonzero value. If a range error occurs because of an underflow (the value is too small), the function returns zero. If an overflow occurs, then the function returns the value of the macro `HUGE_VAL`. Generally, `HUGE_VAL` is the largest value that can be stored in a **double**.

Note that `EDOM` and `ERANGE` are defined in `<errno.h>`, which must be included if you want to check these values.

The math functions are divided into several groups:

- Trigonometric and Hyperbolic Functions
- Exponential and Logarithmic Functions
- Miscellaneous Math Functions

All of the math functions operate on **double** values. However, the ANSI Committee plans to add equivalent functions for **floats** and **long doubles** in the future. The names of these new functions will be the same as the current names with an **f** or **l** appended. Therefore, you should consider these future names as reserved to avoid conflicts at a later date.

The functions `ecvt()`, `fcvt()`, and `gcvt()`, which are available on many systems, were not included in the ANSI Standard because the same functionality can be obtained through `sprintf()`.

A.9.1 Trigonometric and Hyperbolic Functions

With one exception (`atan2()`), all of the trigonometric and hyperbolic functions take a **double** argument and return a **double** result. The general synopsis is

```
#include <math.h>
double func_name( double x );
```

These functions are described in Table A-4 (the `atan2()` function is listed separately). The trigonometric functions use radians, not degrees.

For the *atan2()* function, the synopsis is:

```
#include <math.h>
double atan2( double y, double x );
```

The *atan2()* function returns the principal value of the arc tangent of y/x , using the signs of both arguments to determine the quadrant of the return value. Viewed in terms of a Cartesian coordinate system, the result is the angle between the positive x -axis and a line drawn from the origin through the point (x,y) . The result is in radians and lies between $-\pi$ and π . A domain error occurs if both arguments equal zero.

A.9.2 Exponential and Logarithmic Functions

The following library routines perform exponential and logarithmic functions. Each of these functions returns a **double**.

A.9.2.1 The *exp()* Function

```
#include <math.h>
double exp( double x );
```

The *exp()* function returns the exponential function of x . If the magnitude of x is too large, a range error occurs.

A.9.2.2 The *frexp()* Function

```
#include <math.h>
double frexp( double value, int *exp );
```

The *frexp()* function converts *value* into a fraction multiplied by a power of 2. The fractional part, which is between 0.5 and 1.0, is returned by the function and the exponential value is stored in the object pointed to by *exp*. If the original value of *value* is zero, then both *value* and the object pointed to by *exp* are assigned the value zero.

A.9.2.3 The *ldexp()* Function

```
#include <math.h>
double ldexp( double x, int exp );
```

The *ldexp()* function multiplies the value x by 2 to the power of *exp* and returns the result. If the resulting value is too large to fit in a **double**, a range error may occur.

Table A-4: Trigonometric and Hyperbolic Functions.

Function	Operation
<i>acos()</i>	Returns the principal value of the arc cosine of x . The result lies in the range 0 through π . A domain error occurs if x is less than -1 or greater than 1.
<i>asin()</i>	Returns the principal value of the arc sine of x . The result is in the range $-\pi/2$ through $\pi/2$. A domain error occurs if x is less than -1 or greater than 1.
<i>atan()</i>	Returns the principal value of the arc tangent of x . The result is in the range $-\pi/2$ through $\pi/2$.
<i>cos()</i>	Returns the cosine of x , where x is measured in radians. If x is very large, the result may not be meaningful.
<i>cosh()</i>	Returns the hyperbolic cosine of x . A range error occurs if the magnitude of x is too large.
<i>sin()</i>	Returns the sine of x , where x is measured in radians. If x is very large, the result may not be meaningful.
<i>sinh()</i>	Returns the hyperbolic sine of x . A range error occurs if the magnitude of x is too large.
<i>tan()</i>	Returns the tangent of x , measured in radians. If x is very large, the result may not be meaningful.
<i>tanh()</i>	Returns the hyperbolic tangent of x . A range error occurs if the magnitude of x is too large.

A.9.2.4 The *log()* Function

```
#include <math.h>
double log( double x );
```

The *log()* function returns the natural logarithm of x . If x is negative, a domain error occurs. If x is zero, a range error may occur.

A.9.2.5 The *log10()* Function

```
#include <math.h>
double log10( double x );
```

The *log10()* function returns the base-ten logarithm of x . If x is negative, a domain error occurs. If x is zero, a range error may occur.

A.9.2.6 The *modf()* Function

```
#include <math.h>
double modf( double value, double *iptr );
```

The *modf()* function divides *value* into its integral and fractional parts, each of which has the same sign as *value*. The fractional part is returned and the integral part is stored in the object pointed to by *iptr*.

A.9.2.7 The *pow()* Function

```
#include <math.h>
double pow( double x, double y );
```

The *pow()* function returns the value of *x* raised to the power of *y*. A domain error occurs if *x* is zero and *y* is less than or equal to zero, or if *x* is negative and *y* is not an integer. If *x* is zero and *y* is positive, the result is zero. If *x* is nonzero and *y* is zero, the result is approximately 1.0. If *x* is negative and *y* is an integer, then

```
pow( x, y )
```

is computed as

```
pow ( -x, y )
```

if *y* is even, and as

```
-pow ( -x, y )
```

if *y* is odd. If the result cannot be stored in a **double**, a range error may occur.

A.9.2.8 The *sqrt()* Function

```
#include <math.h>
double sqrt ( double x );
```

The *sqrt()* function returns the nonnegative square root of *x*. If *x* is negative, a domain error occurs.

A.9.3 Miscellaneous Math Functions

A.9.3.1 The *ceil()* Function

```
#include <math.h>
double ceil( double x );
```

The *ceil()* function returns the smallest integer not less than *x*. That is, it rounds toward positive infinity.

A.9.3.2 The *fabs()* Function

```
#include <math.h>
double fabs( double x );
```

The *fabs()* function returns the absolute value of *x*.

A.9.3.3 The *floor()* Function

```
#include <math.h>
double floor( double x );
```

The *floor()* function returns the largest integer not greater than *x*. That is, it rounds toward negative infinity.

A.9.3.4 The *fmod()* Function

```
#include <math.h>
double fmod( double x, double y );
```

The *fmod()* function returns the floating-point remainder of *x* divided by *y*. The result has the same sign as *x*. (Note that technically *fmod()* returns the *remainder*, not the *modulus*, since the sign agrees with *x*, not with *y*.) If the quotient of *x/y* cannot be represented, the behavior is undefined. If *y* is zero, the function returns *x*.

A.10 Nonlocal Jumps

The C library contains two functions—*setjmp()* and *longjmp()*—that enable you to bypass the normal functional call and return procedures. This is particularly useful for dealing with unusual conditions in low-level functions.

The header file for these functions is *<setjmp.h>*, which defines a data type called *jmp_buf*. *jmp_buf* is an array capable of holding the information needed to restore a calling environment.

setjmp() is called once to initialize a *jmp_buf* variable with the current values of the machine's state (e.g., the values of the stack pointer and frame pointer and the registers). When *longjmp()* is called, the machine is reset to the state contained in the *jmp_buf* array. This causes *longjmp()* to return to the place where *setjmp()* was last called with the same *jmp_buf* variable.

A.10.1 The *setjmp()* Function

```
#include <set jmp.h>
int setjmp( jmp_buf env );
```

The *setjmp()* function saves the current environment in its *jmp_buf* argument for later use by the *longjmp()* function. When *setjmp()* returns from an initialization call, it returns zero. When *setjmp()* returns from a *longjmp()* call, it returns a nonzero value.

In some implementations, a *setjmp()* call can appear only in a comparison expression, in which the returned value is compared to an integral constant expression.

A.10.2 The *longjmp()* Function

```
#include <setjmp.h>
void longjmp( jmp_buf env, int val );
```

The *longjmp()* function restores the environment saved by the most recent call to *setjmp()* with the same *env* argument. If there has been no such call, or if the function containing the call to *setjmp()* has terminated (i.e., through a return statement), the behavior is undefined.

After a successful *longjmp()* call, all accessible objects have the same value they had immediately prior to the *longjmp()* call. The only exceptions are dynamic variables that are not **volatile** and have been changed between the calls to *setjmp()* and *longjmp()*. The values of these objects are indeterminate.

If *longjmp()* is invoked from a nested signal handler (that is, from a function invoked as a result of a signal raised during the handling of another signal), the behavior is undefined. In all other interrupt and signal-handling situations, *longjmp()* should execute correctly.

A.10.3 Example

setjmp() and *longjmp()* are typically used to recover from an error deep in the program structure by returning to an earlier state at which the program was functioning properly. In the following example, we initialize *main_loop* at the beginning of the program. Then, whenever *error()* is invoked, the program returns to this initial state.

```

#include <setjmp.h>

jmp_buf main_loop;

main()
{
    if (setjmp( main_loop ))
        printf ( "Restarting. \n" );
    for (;;)
    {
        printf ( "cint> ");
        .
        . /* loop body */
        .
    }
}

void error( char *s )
{
    printf(" Error %s\n", s );
    longjmp ( main_loop );
}

```

Note that the “Restarting” line is not printed the first time *setjmp()* is called because *setjmp()* returns zero when it is explicitly invoked. When *setjmp()* returns via a *longjmp()* call, however, it returns a nonzero value.

A.11 Signal Handling

The C runtime library contains two functions for handling various conditions that may arise during program execution. These functions are a subset of those available in the UNIX library. Both functions make use of a number of macros declared in the header file *<signal.h>*. The macros and their meanings are listed in Table A-5. There is no guarantee, however, that an implementation will use any of these signals (except as the result of explicit calls to *raise()*). In addition to the listed names, the ANSI Committee has also reserved all names that begin with *SIG* for possible future use.

A.11.1 The *signal()* Function

```

#include <signal.h>
void (*signal (int sig, void (*func) (int))) (int);

```

The *func* argument to *signal()* selects one of three methods for subsequent handling of signal number *sig*. If the argument is *SIG_DFL*, then the signal is handled in the default manner. If the argument is *SIG_IGN*, the signal will be ignored. Otherwise, *func* should point to a function that is invoked when the signal occurs.

Table A-5: Signal-Handling Macros.

<i>Macro</i>	<i>Meaning</i>
<i>SIGABRT</i>	Abort Signal – Expands to a positive integral constant expression that is the signal number corresponding to an abnormal termination, such as that indicated by the <i>abort()</i> function.
<i>SIG_DFL</i>	Same as <i>SIG_IGN</i> , except that it specifies that the signal is to be handled in an implementation-defined manner.
<i>SIG_ERR</i>	Same as <i>SIG_IGN</i> except that it specifies that the call to <i>signal()</i> is erroneous.
<i>SIGFPE</i>	Floating-Point Exception Signal—Expands to a positive integral constant expression that is the signal number corresponding to an erroneous arithmetic operation, such as zero divide, or an operation resulting in overflow.
<i>SIG_IGN</i>	Expands to a constant expression of type "pointer to function returning void." It is used as an argument to the <i>signal()</i> function, in place of a function address, to specify that a given signal should be ignored.
<i>SIGILL</i>	Illegal Instruction Signal—Expands to a positive integral constant expression that is the signal number corresponding to detection of an invalid function image.
<i>SIGINT</i>	Interrupt Signal—Expands to a positive integral constant expression that is the signal number corresponding to receipt of an interactive attention signal.
<i>SIGSEGV</i>	Segment Violation Signal—Expands to a positive integral constant expression that is the signal number corresponding to an invalid access to storage.
<i>SIGTERM</i>	Termination Signal—Expands to a positive integral constant expression that is the signal number corresponding to a termination request sent to the program.

Even if *func* points to a signal-handling function, the system will still execute the equivalent of

```
signal ( sig, SIG_DFL );
```

before invoking

```
(*func) ( sig );
```

The function *func* may terminate in several ways, including calls to *abort()*, *exit()*, or *longjmp()*. If it returns through a **return** statement, and the return value is *SIGFPE* or any other implementation-defined value

that corresponds to an exception, the behavior is undefined. Otherwise, the program resumes execution at the point it was interrupted. If the requested change can be honored, *signal()* returns the value of *func*. Otherwise, it returns *SIG_ERR* and sets *errno* to indicate an error.

The functions in the C runtime library are not guaranteed to be reentrant, so they may not be used reliably with a signal handler that returns.

At program start-up, the equivalent of

```
signal( sig, SIG_IGN )
```

may be executed for some signals in an implementation-defined manner.

For all other signals the equivalent of

```
signal ( sig, SIG_DFL );
```

is executed.

A.11.2 The *raise()* Function

```
#include <signal.h>
int raise( int sig );
```

The *raise()* function sends the signal *sig* to the executing program. If successful, *raise()* returns zero; if unsuccessful, it returns a nonzero value.

A.12 Variable Argument Lists

The C library contains several tools for writing functions that can accept a variable number of arguments in a portable fashion. Without using these tools, you need to know the stack implementation of a particular compiler to write a variable argument function. The macros and function discussed in this section enable you to avoid the compiler internals.

To declare a function capable of accepting a variable number of arguments, use the “...” prototype syntax. For example, the following defines a function *f1()* that will always be passed at least two arguments, but might be passed more.

```
void f1 ( int arg1, int arg2, ... )
{
    :
}
```

Within *f1()*, you would use the variable argument macros and functions to process all of the arguments to *f1()*. Wherever *f1()* is invoked, there must be a function allusion of the form

```
void f1 ( int, int, ... );
```

The *<stdarg.h>* file contains definitions of two macros, one function, and one data type. The macros are *va_start*, *va_arg*, the function is

va_end, and the type is *va_list*. *va_list* defines an array type suitable for holding information needed by *va_arg* and *va_macro*. In the following discussion, the name of the array with this type is *ap*.

A.12.1 The *va_start* Macro

```
#include <stdarg.h>
void va_start( va_list ap, parmN );
```

The *va_start* macro initializes the array *ap* for subsequent use by *va_arg* and *va_end*. It should be invoked before any arguments are processed. The *parmN* argument should be the name of the rightmost argument before the three dots. Continuing our example of *f1()*, the *va_start* invocation would be

```
va_start ( ap, arg2 );
```

A.12.2 The *va_arg* Macro

```
#include <stdarg.h>
type va_arg( va_list ap, type );
```

The *va_arg* macro expands to an expression that has the type and value of the next argument. It should be invoked once for each argument. The argument *ap* should be the same argument initialized by the *va_start* macro. The parameter *type* should be the type of the argument (after default conversions to **int**, **unsigned int**, or **double**). For example, if all the arguments to *f1()* are integers, you would write

```
arg1 = va_arg( ap, int );
arg2 = va_arg( ap, int );
```

to get the value of each argument. If the type specified in *va_arg* does not match the actual argument type, the behavior is undefined.

A.12.3 The *va_end()* Function

```
#include <stdarg.h>
void va_end( va_list ap );
```

The *va_end()* function should be invoked after all of the arguments have been processed to facilitate a normal return from the function. If it is not invoked, the behavior is undefined.

A.12.4 Example

The following function accepts from 1 to 20 integer arguments, which it processes and stores in elements of the array *args[]*. The first argument specifies the total number of arguments in the call.

Program
A-1

```
# include <stdarg.h>
#define MAX_ARGS 20

void f1( int arg_num, ... )
{
    va_list ap;
    int arg[MAX_ARGS];
    int array_element = 1;

    va_start( ap, arg_num );
    while (arg_num)
        arg[array_element++] = va_arg( ap, int );
    va_end( ap );
}
```

A.13 I/O Functions

This section describes each standard I/O function in detail. See Chapter 12 for more general information about how to perform I/O using the C library.

A.13.1 The *clearerr()* Function

```
#include <stdio.h>
void clearerr( FILE *stream );
```

The *clearerr()* function clears the end-of-file and error indicators associated with the specified stream. Use *ferror()* and *feof()* to see whether these indicators are set. The only other time these indicators are cleared is when the file is opened or when a *rewind()* function is executed. The *clearerr()* function does not return a value.

A.13.2 The *fclose()* Function

```
#include <stdio.h>
int fflush( FILE *stream );
```

The *fclose()* function closes the file associated with the specified stream and disassociates the stream from the file. Before closing the file, *fclose()* flushes the associated buffers. If the buffers had been automatically allo-

cated, they are deallocated. Whenever the *exit()* function is invoked, it calls *fclose()* for any open streams.

The *fclose()* function returns zero if it successfully closes the stream, or nonzero if an error occurs. If the stream is already closed, *fclose()* returns nonzero.

On some operating systems, it is impossible to create a file without writing something to it. Programs that rely on a file being created, therefore, should write something to the associated stream before closing it.

A.13.3 The *feof()* Function

```
#include <stdio.h>
int feof( FILE *stream );
```

The *feof()* function tests the end-of-file indicator for the specified stream to see whether an end-of-file was encountered in a previous read or write operation. If the indicator is set, *feof()* returns a nonzero value; otherwise it returns zero. Note that *feof()* does not reset the error indicator so that repeated calls to *feof()* will report the same condition over and over. To reset the error indicator, use *clearerr()*.

A.13.4 The *ferror()* Function

```
#include <stdio.h>
int ferror( FILE *stream );
```

The *ferror()* function tests the error indicator for the specified file to see whether an error has occurred on a previous read or write operation. If the indicator is set, *ferror()* returns a nonzero value; otherwise it returns zero. Note that *ferror()* does not reset the error indicator so that repeated calls to *ferror()* will report the same error over and over. To reset the error indicator, use *clearerr()*.

A.13.5 The *fflush()* Function

```
#include <stdio.h>
int fflush( FILE *stream );
```

The *fflush()* function empties the buffer associated with the specified stream, causing any data in the buffer to be written to the destination file or device. The stream remains open. If the call is successful, *fflush()* returns zero; otherwise it returns a nonzero value.

A.13.6 The *fgetc()* Function

```
#include <stdio.h>
int fgetc( FILE *stream );
```

The *fgetc()* function fetches the next character from the specified stream, returns the value after converting it to an **int**, and advances the associated file position indicator. Successive calls to *fgetc()* return successive characters from the stream. If an end-of-file is encountered, or if an error occurs, *fgetc()* returns *EOF*. Use *feof()* or *ferror()* to determine whether an error or end-of-file occurred.

A.13.7 The *fgetpos()* Function

```
#include <stdio.h>
int fgetpos( FILE *stream, fpos_t *pos );
```

The *fgetpos()* function stores the current file position indicator in the object pointed to by *pos*. It is similar to *ftell()*, except that the file position indicator value is stored in an object of type *fpos_t*, rather than being returned as a **long int**. The value stored contains implementation-defined information that can only be used by *fsetpos()* to reposition the file position indicator to its position at the time of the *fgetpos()* call. See the description of *fsetpos()* for more information.

If successful, *fgetpos()* returns zero. On a failure, *fgetpos()* returns a nonzero value and sets *errno* to an implementation-defined value.

A.13.8 The *fgets()* Function

```
#include <stdio.h>
char *fgets( char *s, int n, FILE *stream );
```

The *fgets()* function reads characters from the specified stream and assigns them to the array identified by *s*. The stream must be open with read access. Characters are read until a newline or end-of-file is encountered, or until *n - 1* characters have been fetched, whichever comes first. Unlike the *gets()* function, *fgets()* includes the terminating newline in the array. *fgets()* then appends a null character after the last character assigned, so that the maximum number of array elements used is *n*.

If successful, *fgets()* returns the pointer *s*. If an end-of-file is encountered before any characters are read, *fgets()* leaves the array untouched and returns a null pointer. If an error occurs, a null pointer is returned, but the contents of the array are unpredictable.

A.13.9 The *fopen()* Function

```
#include <stdio.h>
FILE *fopen( const char *filename,
             const char *mode );
```

The *fopen()* function opens a file identified by *filename* and associates a stream with the file. The second argument is a pointer to a character string that identifies the file access type. Table A-6 shows the legal values for the argument *mode*.

Many of these access types were invented by the ANSI Committee, so they may not be implemented on older compilers. The traditional access types, documented in K&R, are "r", "w", and "a". The corresponding *update modes*, "r+", "w+", and "a+", have also been in existence for some time. The types for accessing binary data (those with a **b** in them) are new and reflect ANSI's efforts to develop a consistent library of functions. Formerly, I/O to binary conversion was performed through a set of UNIX-derived functions that paralleled the standard text I/O functions. Now they are merged into one group.

The "ab", "ab+", and "a+b" access modes may initially position the file position indicator beyond the last data written, due to null padding. The ANSI Standard also leaves it open for compilers to support additional access modes beyond the ones listed here.

Opening a file with one of the append modes ("a", "a+", "ab", or "a+b") forces all subsequent writes to occur at the current end-of-file, regardless of previous calls to *fseek()*. After each write operation, the file position indicator is repositioned to the end of the file and the buffer is flushed.

Opening a file in read mode (where **r** is the first character of the mode argument) fails if the file does not exist or cannot be read.

One peculiarity of the update modes (which stems from the fact that I/O is buffered) is that you cannot write to a file and then read from it, or vice versa, without an intervening *fseek()*, *fsetpos()*, *rewind()*, or *fflush()* call (unless the read or write operation encounters an end-of-file).

The *fopen()* function returns a pointer to a structure of type *FILE*. This pointer, called a *file pointer*, is then used to access the file in subsequent I/O operations. If an error occurs while opening the file, *fopen()* returns a null pointer.

Table A-6: The `fopen()` Modes.

<i>Mode</i>	<i>Meaning</i>
"r"	Open an existing text file for reading.
"w"	Create a new text file for writing or truncate an existing file.
"a"	Open a text file in append mode; writing to the file occurs at end-of-file marker.
"rb"	Open a binary file for reading.
"wb"	Create a new binary file for writing or truncate an existing binary file.
"ab"	Open or create a binary file in append mode; writing occurs at end-of-file marker.
"r+"	Open an existing text file for reading and writing.
"w+"	Create a new text file for reading and writing or truncate an existing file.
"a+"	Open an existing file or create a new one in append mode; writing occurs at end-of-file.
"r+b" or "rb+"	Open a binary file for reading and writing.
"w+b" or "wb+"	Create a new binary file for writing or truncate an existing binary file.
"a+b" or "ab+"	Open an existing binary file or create a new one for reading and writing in append mode; writing occurs at end-of-file.

A.13.10 The `fprintf()` Function

```
#include <stdio.h>
int fprintf( FILE *stream, const char *format, ...
            );
```

The `fprintf()` function enables you to send formatted output to a file. This function is equivalent to `printf()`, except that it takes one additional argument, *stream*, which lets you specify a stream (the `print()` function automatically writes to the standard output stream *stdout*). See the description of `printf()` for more information.

A.13.11 The *fputc()* Function

```
#include <stdio.h>
int fputc( int c, FILE *stream );
```

The *fputc()* function writes a single character to the specified stream and advances the associated file position indicator. Note that the character is passed as an *int*, but *fputc()* converts it to an **unsigned char** before outputting it. *fputc()* returns *EOF* if an error occurs; otherwise it returns the character written.

The ANSI Standard guarantees that *fputc()* will not be implemented as a macro. *putc()* is an equivalent function that may be implemented as a macro. See Chapter 12 for more information about *putc()* and *fputc()*.

A.13.12 The *fputs()* Function

```
#include <stdio.h>
int fputs( const char *s, FILE *stream )
```

The *fputs()* function writes the array identified by the pointer *s* to the specified stream. Characters from the array are written up to, but not including, the terminating null character. Note that *fputs()* does not insert a newline as *puts()* does. Also note that the string must have a terminating null character or *fputs()* will output successive bytes from memory indefinitely. If successful, *fputs()* returns zero; otherwise it returns a nonzero value.

A.13.13 The *fread()* Function

```
#include <stdio.h>
int fread( void *ptr, size_t size, int nelem,
           FILE *stream );
```

The *fread()* function is used to read a block of binary or text data into an array. The array is identified by *ptr*. The argument *nelem* specifies the number of elements to read, and *size* specifies the size of each element in bytes. Normally, the size is computed by using the *sizeof* operator. For example,

```
fread( arr, sizeof(*arr), 100, fp );
```

reads 100 elements from the stream identified by *fp* and stores the results in an array called *arr*. The array must be large enough to hold the data.

The *fread()* function concludes when it reads in the specified number of bytes, it encounters an end-of-file, or a read error occurs. In all three cases, *fread()* returns the number of bytes read. If the returned value is less than the number of bytes specified in the call, use *ferror()* or *feof()* to determine why *fread()* ended prematurely. After a *fread()* call, the file position indica-

tor is positioned just after the last byte read. You can reposition it with an *fseek()* or *rewind()* call.

A.13.14 The *freopen()* Function

```
#include <stdio.h>
FILE *freopen(const char *filename,
               const char *mode, FILE *stream );
```

The *freopen()* function is used to associate an existing stream with a different file. Normally it is used to redirect the standard streams, *stdin*, *stdout*, and *stderr*. First, *freopen()* closes the file associated with the stream; then it opens the file identified by *filename* and associates the stream to it. The mode argument serves the same role as in an *fopen()* function. If *freopen()* encounters an error, it returns a null pointer; otherwise it returns the value of the file pointer (the third argument).

A.13.15 The *fscanf()* Function

```
#include <stdio.h>
int fscanf(FILE *stream, const char *format, ... );
```

The *fscanf()* function enables formatted data to be read into variables. It is equivalent to *scanf()*, except that it takes one additional argument, *stream*, which lets you specify an input stream (the *scanf()* function automatically reads from *stdin*). See the description of *scanf()* for more information.

A.13.16 The *fseek()* Function

```
#include <stdio.h>
int fseek( FILE *stream, long offset, int ptrname
           );
```

The *fseek()* function enables you to move the file position indicator in order to perform random access on a file. The offset refers to the number of bytes from a fixed position specified by *ptrname*. *ptrname* can have one of three values represented by macros defined in *<stdio.h>*:

SEEK_SET Offset from the beginning of the file.

SEEK_CUR Offset from the current value of the file position indicator.

SEEK_END Offset from the end of the file.

Note that the offset can be negative. However, if you attempt to move the file position indicator before the beginning of the file, the results are unpredictable.

The *fseek()* function has somewhat different semantics depending on whether the stream is open in binary or text mode. For binary streams, the name *SEEK_END* may not have meaning. For text streams, *ptrname* must be *SEEK_SET* and the offset must be either zero or a value returned by a previous call *ftell()*.

The *fseek()* function undoes the effects of an *ungetc()* function. It also resets the end-of-file flag. *fseek()* returns zero if it is successful; if unsuccessful, it returns a nonzero value.

A.13.17 The *fsetpos()* Function

```
#include <stdio.h>
int fsetpos( FILE *stream, const fpos_t *pos );
```

The *fsetpos()* function is designed to be used in conjunction with *fgetpos()* to move the file position indicator to the spot specified by the object pointed to by *pos*. *pos* must be a value returned by an earlier call to *fgetpos()*. The *fgetpos()* and *fsetpos()* functions should be used instead of *ftell()* and *fseek()* when the file position indicator value is too large to fit in a *long int*. The *fpos_t* data type should be defined by each implementation to be large enough to hold the largest possible file position indicator value.

The *fsetpos()* function clears the end-of-file flag for the specified stream and undoes the effects of any previous calls to *ungetc()* on the same stream. After an *fsetpos()* call, the next operation on the stream can be either input or output.

If successful, *fsetpos()* returns zero. If it fails, *fsetpos()* returns a nonzero value and sets *errno* to an implementation-defined nonzero value.

A.13.18 The *ftell()* Function

```
#include <stdio.h>
long ftell( FILE *stream );
```

The *ftell()* function returns the current value of the file position indicator. For binary streams, this is the number of bytes from the beginning of the file. For text streams, *ftell()* returns an implementation-defined value that is suitable for use as an offset value in an *fseek()* function. Using the value in an *fseek()* call repositions the file position indicator to its position at the time of the *ftell()* call.

The *ftell()* function can fail for at least two reasons:

- The stream is associated with a terminal, or other file type, for which the concept of a file position indicator is meaningless.
- The current value of the file position indicator cannot be represented in a *long int* (see the description of *fsetpos()*).

If either of these failures occurs, *ftell()* returns *-1L* and sets *errno* to an implementation-defined nonzero value.

A.13.19 The *fwrite()* Function

```
#include <stdio.h>
int fwrite( const void *ptr, size_t size,
            size_t nmemb, FILE *stream );
```

The *fwrite()* function writes the array pointed to by *ptr* to the specified stream. It writes *nmemb* elements, where each element is *size* bytes long. Normally, *size* is computed by using the *sizeof* operator. For example,

```
fwrite( arr, sizeof(*arr) , 100, s );
```

writes 100 elements from array *arr* to stream *s*. Note that *fwrite()* continues fetching elements from the array until *nelem* elements have been read, even if this means going past the end of the array. The size of the array must be at least as long as *nelem* times *size*. After the *fwrite()* call has completed, the file position indicator is positioned just after the last character written. *fwrite()* does not modify the array in any way.

fwrite() returns the number of elements written. Assuming no error occurs, this will be the same as *nmemb*.

A.13.20 The *getc()* Function

```
#include <stdio.h>
int getc( FILE *stream );
```

The *getc()* function reads the next character from the specified stream and returns it as an *int*. *getc()* is equivalent to *fgetc()* except that it may be implemented as a macro instead of a function. If the next character in the stream is an end-of-file, or if an error occurs during the read operation, *getc()* returns *EOF*. Use *ferror()* or *feof()* to determine whether an end-of-file or error occurred.

A.13.21 The *getchar()* Function

```
#include <stdio.h>
int getchar( void );
```

The *getchar()* function is equivalent to

```
getc( stdin )
```

which is, in fact, how it is implemented by most compilers. It returns the next character from the standard input stream, or *EOF* if an end-of-file or error occurs. Use *ferror()* or *feof()* to determine whether an error or end-of-file occurred.

A.13.22 The *gets()* Function

```
#include <stdio.h>
char *gets( char *s );
```

The *gets()* function reads characters from the standard input stream (*stdin*) and assigns them to the character array identified by *s*. Characters are read and assigned until a newline or end-of-file is encountered. *gets()* is similar to

```
fgets( s, n, stdin )
```

where *n* is a large number. Note, however, that unlike *fgets()*, *gets()* does not allow you to specify a maximum number of characters to read. *gets()* and *fgets()* also differ in the way they handle newlines. If *gets()* ends by reading a newline, it absorbs the newline (i.e., positions the file position indicator after the newline) but does not assign the newline to the array. In contrast, *fgets()* includes the newline in the array. Both *gets()* and *fgets()* append a null character after the last character assigned to the array.

If an end-of-file is encountered before any characters are read, *gets()* returns a null pointer and leaves the array untouched. If an error occurs during the read operation, a null pointer is returned and the contents of the array are unpredictable. Otherwise, if *gets()* concludes successfully, it returns *s*.

A.13.23 The *perror()* Function

```
#include <stdio.h>
char *perror( const char *s );
```

The *perror()* function returns an error message corresponding to the value of *errno*. If *s* is not a null pointer, *perror()* writes the string pointed to by *s* to *stderr*, then writes a colon, and then the error message that matches the current value of *errno*. If *s* is a null pointer, *perror()* returns a pointer to the error message string but performs no output. Note that *perror()* does not reset *errno*, so you should explicitly reset *errno* to zero after each *perror()* call.

A.13.24 The *printf()* Function

```
#include <stdio.h>
int printf( const char *format, ... ) ;
```

The *printf()* function writes formatted data to the standard output stream (*stdout*). The first argument is a character string that may contain text and format control expressions called *conversion specifiers*. The remaining arguments represent the actual data to be written. For each data argument, there should be one, and only one, conversion specifier in the format string which defines how the data is to be output. Conversion specifiers and arguments are associated in the order in which they occur. If there are more data arguments than conversion specifiers, the remaining data expressions are evaluated and then ignored. If there are more format specifiers than data arguments, the behavior is implementation-defined.

The *printf()* function is closely related to the *fprintf()*, *sprintf()*, *vfprintf()*, and *vfprintf()* functions. They all obey the same formatting rules. The only difference is that *printf()* always writes to *stdin*, whereas *fprintf()* and *sprintf()* allow you to specify an output stream or internal buffer, respectively. *vfprintf()* and *vfprintf()* are identical to *printf()* and *fprintf()* except that the argument list is replaced by a predeclared argument array. All of these functions return the number of data arguments written, or a negative number if an error occurs.

Each conversion specifier starts with a percent sign (%) and is followed by optional *format modifiers* and a *conversion character*. The conversion character specifies the type of data (integer, floating-point, or character). The corresponding data argument must match this type. The format modifiers control such things as field width, left and right justification, and the padding character. The following example shows a simple *printf()* call.

```
int j = 5;
printf( "The value of j is: %d\nThe value of j\
      squared is: %6d", j, j*j ) ;
```

The output is

```
The value of j is: 5
The value of j squared is:    25
```

Note that there is no explicit separator between text and conversion specifiers. The *printf()* function knows it has reached the end of a conversion specifier when it reads a conversion character. In the example, the conversion character is *d*, which directs *printf()* to write an integer in decimal format. The conversion specifier *%d* is associated with argument *j*, and *%6d* is associated with *j*j*. The 6 following the percent sign is a field width. Note in the output that the value is right-justified by default and padded with spaces on the left. The legal conversion characters are shown in Table A-7.

Table A-7: *printf()* Conversion Characters.

Conversion Character	Effect
d, i, o, u, x, X	Used to format integer output. d and i output the data item in decimal form; o prints the data in octal form; u prints the unsigned value of the data item; x and X print the value in hexadecimal format. x uses the lowercase letters abcdef, while X uses ABCDEF. These formats output as many digits as are required to represent the number. Just before one of these conversion characters, you may enter an h , l , or L . The h signifies that the corresponding data item is a short int or unsigned short int , and the l and L signify that the data item is a long int or unsigned long int .
f	Prints floating-point values in decimal notation (i.e., 35.734). The precision designates the number of digits to appear after the decimal point. The default precision is 6. If the precision equals zero, the decimal point is not printed. As long as there is a decimal point, however, there must be at least one digit to the left of it, even if the value is less than one (i.e., 0.3411).
e, E	Outputs a floating-point value using scientific notation (i.e., 3.67e+08). There is always one digit to the left of the decimal point. The number of digits to the right of the decimal point is determined by the precision. The default precision is 6. If the precision is zero, the decimal point is not printed. The exponent value contains at least two digits and as many digits thereafter as are needed to represent the datum. The letter separating the decimal value from the exponent is either e or E depending on which conversion character you use.

Table A-7 (continued)

<i>Conversion Character</i>	<i>Effect</i>
g, G	Uses either f or e (E if G is specified), depending on the value of the datum. If the value would require an exponent less than -4 or greater than the precision, then e (or E) is used. Otherwise f is used. The precision specifier has the same effect it has for the f , e , and E conversions. Trailing zeros are removed from the result, and a decimal point is printed only if it is followed by a digit.
C	Prints a character. Because the data argument is passed as an int , <i>printf()</i> prints the least significant byte.
s	Prints a string of characters. The data argument should be a pointer to a null-terminated array. For this conversion format, the precision is interpreted as the maximum number of characters to output. Any additional characters in the string are ignored. If you do not specify a precision, all characters up to, but not including, the terminating null character are printed.
P	The corresponding data argument should be a pointer to an object of type void . The value of the pointer is converted into a sequence of characters in an implementation-defined manner.
N	Records the number of data items written so far. The corresponding data argument should be a pointer to an int . <i>printf()</i> fills the int with the number of objects printed so far.
%	The sequence % % outputs a percent sign.

The following program illustrates the default format for each conversion character. The next sections describe how to change the default by specifying a minimum field width, a precision, left justification, and zero as the pad character.

Program
A-2

```
# include <stdarg.h>
#define MAX_ARGS 20
void f1( int arg_num, ... )
{
    va_list ap;
    int arg[MAX_ARGS];
    int array_element = 1;
    va_start( ap, arg_num );
    while (arg_num-->0)
        arg[array_element++] = va_arg( ap, int );
    va_end( ap );
}
```

The output is:

```
%d %u %o %x
-25 4294967271 31 19
%c %s
A String
%f %e %g
234.567800 0.234568e+03 234.568
```

Flag Characters—There are a number of optional format modifiers that may come before the conversion character. The first, called a *flag character*, can be any of the characters shown in Table A-8.

Table A-8: printf() Flag Characters.

Flag Character	Meaning
-	Specifies left justification.
+	Causes all numeric data to be prefixed with a plus or minus sign. The default, which this character overrides, is to print a minus sign for negative numbers but no plus sign for positive numbers.
Space	Causes negative numbers to be prefixed with a minus sign and positive numbers to be prefixed with a space. (The default is no space for positive numbers.)
#	This modifier has various meanings depending on what conversion character is specified. For c , d , I , s , and u , this flag has no effect. For o conversions, the # flag causes the value to be prefixed with a zero (the precision is widened if necessary). For x and X conversions, the value is prefixed with 0x or 0X . For e , E , f , g , and G conversions, the # causes the result to contain a decimal point, even if the precision is zero. For g and G conversions, trailing zeros will not be removed from the result, as they are normally.

The following program shows the effects of the flag characters in various situations.

Program
A-3

```
#include <stdio.h>
#include <stdlib.h>

int main( void )
{
    printf( "%5d\t%5d\n", 25 );
    printf( "%-5d\t%5d\n", 25 );
    printf( "%+5d\t%+5d\n\n", 25 );
    printf( "%o\t%o\n", 25 );
    printf( "%#o\t%#o\n", 25 );
    printf( "%x\t%x\n", 25 );
    printf( "%#x\t%#x\n\n", 25 );
    printf( "%5.0f\t%5.0f\n", 25.0 );
    printf( "%#5.0f\t%#5.0f\n\n", 25.0 );
    printf( "%+-5d\t%+5d\n", 25 );
    printf( "%+#5.0f\t+#5.0f\n\n", 25.0 );
    exit( 0 );
}
```

The output is

```
%5d 25
%-5d 25
%+5d +25
%o 31
%#o 031
%x 19
%#x 0x19
%5.0f 25
%#5.0f 25.
%+-5d +25
%+#5.0f +25.
```

Note that the flags are not mutually exclusive. You can combine them, as shown in the last two *printf()* calls.

Minimum Field Width—The next optional format component is an optional *minimum field width*. This is a decimal constant that represents the minimum number of characters to output. If the data item requires fewer characters, it is padded on either the left or the right until the minimum width is reached. (The default is to pad on the left, but you can specify right padding with the *left adjustment flag*.) The default pad character is a space, but you can make it a zero by making the first digit of the minimum field width a zero. If the value requires more characters than the minimum field width, the field is expanded to accommodate the data. The value is never truncated.

Another way to express the minimum field width is through a dynamic variable. In this case, you enter an asterisk (*), which informs the *printf()* function to use the next data argument as the field width. The following examples show the effects of several minimum field width specifications.

Program
A-4

```
#include <stdio.h>
#include <stdlib.h>

int main( void )
{
    printf( "%10d\t%10d\n", 25 );
    printf( "%010d\t%010d\n", 25 );
    printf( "%1d\t%1d\n", 25 );
    printf( "%*f\t%*f\n", 5, 33.87 );
    printf( "%7f\t%7f\n", 33.87 );
    exit( 0 );
}
```

The output is

```
%10d 25
%010d 0000000025
%1d 25
%*f 32.87000
%7f 32.8700000
```

Note that the 5 in the fourth *printf()* call corresponds to the asterisk in the format specifier and represents the minimum field width for the next data item.

Precision Specifier—The next optional component is a *precision specifier*, which is designated by a period followed by a decimal constant. For floating-point values, the precision determines the number of digits to appear after the decimal point. For integer values, the precision specifier has the same meaning as the minimum field width specifier and overrides that specifier. For strings, the precision specifier denotes the maximum number of characters to print.

The program below illustrates the effects of several precision specifications.

Program
A-5

```
#include <stdio.h>
#include <stdlib.h>

int main( void )
{
    printf( "%5d\t%5d\n", 25 );
    printf( "%5.3d\t%5.3d\n", 25 );
    printf( "%.3d\t%.3d\n", 25 );
    printf( "%4.3f\t%4.3f\n", 23.45 );
    printf( "%4.3f\t%4.3f\n", 23.456789 );
    printf( "%4.3e\t%4.3e\n", 23.456789 );
    printf( "%.10s\t%.10s\n", "Print only the first",
            " ten characters." );
    exit( 0 );
}
```

The output is

```
%5d 25
%5.3d 025
%.3d 025
%4.3f 23.450
%4.3f 23.457
%4.3e 0.235e+02
%.10s Print only
```

Note that for integer conversions, the field is padded with zeroes on the left until the precision length is reached. For floating-point values, if the true value cannot be expressed in the number of digits reserved by the precision, it is rounded. Rounding can occur either up or down, depending on the implementation.

Short and Long Specifiers—Just before the conversion character, you may enter an **h**, **l**, or **L**. The **h** signifies that the corresponding data item is a **short int** or **unsigned short int**, and the **l** signifies that the data item is a **long int** or **unsigned long int**. These prefixes may be used only for integer specifiers. An **L** signifies that the corresponding argument is a **long double**. Because integral arguments are converted to **int**, and floating-point arguments are converted to **double** when they are passed to *printf()*, these prefixes ensure that arguments are cast back to their original type. If used for incompatible types, these flags are ignored.

A.13.25 The *putc()* Function

```
#include <stdio.h>
int putc( int c, FILE *stream );
```

The *putc()* function writes a character to the specified stream. It is equivalent to *fputc()* except that it may be implemented as a macro instead of a function. *putc()* returns *EOF* if an error occurs; otherwise, it returns the character written. Note that both the argument and the returned value are **ints**. *putc()* outputs the least significant byte of the argument.

A.13.26 The *putchar()* Function

```
#include <stdio.h>
int putchar( int c );
```

The *putchar()* function writes its argument to the standard output stream (*stdout*) and returns the character written. If an error occurs, *putchar()* returns *EOF*.

The expression

```
putchar( c )
```

is equivalent to

```
putc( c, stdout )
```

A.13.27 The *puts()* Function

```
#include <stdio.h>
int puts( const char *s );
```

The *puts()* function writes the string pointed to by *s* to the standard output stream (*stdout*) and appends a newline character to the output. The terminating null character in the array is not written. The call

```
puts( s )
```

is equivalent to

```
fputs( s, stdin )
```

except that *fputs()* does not append a newline character. If *puts()* is successful, it returns zero; if an error occurs, it returns a nonzero value.

A.13.28 The *remove()* Function

```
#include <stdio.h>
int remove( const char *filename );
```

The *remove()* function is used to delete the file identified by *filename*. If you try to delete a file that is open, the results are implementation-defined. *remove()* returns zero if successful or a nonzero value in the event of a failure. This is a new function that is not included in older C and UNIX libraries.

A.13.29 The *rename()* Function

```
#include <stdio.h>
int rename( const char *old, const char *new );
```

The *rename()* function changes the name of a file from the name pointed to by *old* to the name pointed to by *new*. After execution, the name identified by the pointer *old* no longer exists. If the file identified by the pointer *old* is open, the effect is implementation-defined. Likewise, if the name pointed to by *new* already exists, the results are implementation-defined. *rename()* returns zero when it succeeds and a nonzero value when it fails.

A.13.30 The *rewind()* Function

```
#include <stdio.h>
void rewind( FILE *stream );
```

The *rewind()* function moves the file position indicator for *stream* to the beginning of the file. The file identified by *stream* should be open on a *rewind()* call. The function call

```
rewind( s )
```

is equivalent to

```
(void) fseek ( s, 0L, SET_SEEK )
```

except that the *rewind()* function clears the end-of-file and error indicators for the stream and does not return a value.

A.13.31 The *scanf()* Function

```
#include <stdio.h>
int scanf( FILE *stream, const char *format, ... );
```

The *scanf()* function reads data from *stdin* in a form specified by a format string. The syntax and semantics of *scanf()* are, to a large extent, the reverse of the *printf()* function. However, there are enough differences that you should not assume that conversion specifiers behave identically in both functions.

As with the *printf()* function, the first argument to *scanf()* is a format string. There can be any number of data arguments following the format string. Each one should be the address of a variable where the data is to be stored. The data type of each pointer argument must match the type specified by the corresponding conversion character.

The format string consists of literal characters interspersed with conversion specifiers. A conversion specifier begins with a percent sign followed by optional conversion modifiers and a required conversion character. It designates how many characters to read and how to interpret them. Characters other than a conversion string, a space, a newline, or a vertical tab must match characters in the input stream. A space, horizontal tab, or newline character occurring in the format string causes *scanf()* to skip over characters up to the next nonspace character. For example, the statement

```
scanf( "Value: %d", &n );
```

directs *scanf()* to skip over leading spaces, to read the literal "Value:", and then to read a decimal constant and store it in the object pointed to by *n*. If the first nonspace characters are not "Value:", the function will fail and the results are unpredictable.

The legal conversion characters and their meanings are shown in

Any conversion character may be preceded by a *maximum field width* or an *assignment suppression flag*. The field width is written in the form of a decimal digit and directs the *scanf()* function not to read any more than the specified number of characters for that particular item. The assignment suppression flag is an asterisk (*), which causes *scanf()* to read the data item but not to assign it to a variable. Consequently, do not enter a corresponding data pointer for a conversion specification with an asterisk.

Table A-9: `scanf()` Conversion Characters.

Conversion Character	Effect
d	Reads a decimal integer. The corresponding data argument should be a pointer to an integer.
I	Reads a decimal integer, possibly with a prefix and/or suffix. Legal prefixes are a minus sign (-), a plus sign (+), 0x or 0X to denote a hexadecimal constant, and 0 to denote an octal constant. Legal suffixes are u or U to denote an unsigned integer and I or L to denote a long integer. The corresponding data argument should be the address of a variable with the appropriate type.
O	Reads an octal constant. Even if the constant does not begin with a 0 , it is treated as an octal value. The corresponding argument should be the address of an integer variable.
U	Reads an unsigned decimal constant. The corresponding data argument should be the address of an integer variable.
x, X	Reads a hexadecimal constant. The corresponding data argument should be the address of an integer variable.
e, E, f, g, G	Reads a floating-point constant. The corresponding data argument should be a pointer to a float . (Use an l prefix to indicate that the corresponding argument is a pointer to a double and an L prefix to indicate that the corresponding argument is a pointer to a long double .) The floating-point constant may appear in either decimal or scientific form. These format characters may be used interchangeably.
S	Reads a character string. Characters are read until a space, horizontal tab, or newline is encountered. The corresponding argument should be a pointer to an array of chars . Each character in the string is loaded into the subsequent array element up to, but not including, the terminating null character. The <code>scanf()</code> function automatically adds a null character as the last character of the string. Because there is no bounds checking in C, it is your responsibility to ensure that the character array is long enough to hold the input string.

Table A-9: (continued)

Conversion Character	Effect
c	Reads the next character in the stream. It does not skip over spaces, null characters, or tabs. To read the next nonspace character, use <code>%1s</code> . If the <code>c</code> conversion character is preceded by a field width, then the specified number of characters are read and the corresponding data argument should be a pointer to an array of <code>chars</code> . Otherwise, the data argument can be a pointer to a single <code>char</code> .
p	Reads a pointer. The actual representation of the pointer value in the input field is implementation-defined, but it should be the same as that produced by the <code>%p</code> conversion of <code>printf()</code> . The corresponding data argument must be a pointer to a pointer to <code>void</code> .
n	Records the number of characters read thus far by this <code>scanf()</code> call. No characters are read for this conversion character. The corresponding data argument should be a pointer to an integer.
[<i>scan list</i>]	Reads a character string. If the first character in the <i>scan list</i> (a list of characters) is not a circumflex (^), then characters are read from the input stream until a character is read that is <i>not</i> a member of the <i>scan list</i> . If the first character is a circumflex, then the scan list serves as a terminating set— <code>scanf()</code> reads characters from the input stream until it encounters one of the characters in the list. The corresponding data argument should be a pointer to an array of <code>chars</code> . The array is loaded with the characters read. <code>scanf()</code> automatically appends a null character after the last character.
%	Reads a percent sign. No assignment occurs.

The `scanf()` function continues reading characters from the input stream until the format string is exhausted, or an end-of-file is encountered, or a conflict occurs. A conflict can occur whenever the next character in the stream does not match the conversion specifier. For example, the next character might be a letter, whereas the conversion specifier indicates a numeric value. A conflict also occurs if the format string contains a string literal that is not matched by the next character in the input stream. Regardless of whether a conflict occurs or whether `scanf()` completes successfully, it returns the number of data items assigned. However, if an end-

of-file is encountered before a conversion or conflict takes place, *scanf()* returns *EOF*.

The following examples show several ways to read an input stream using *scanf()*. Assume that the input stream for all three examples is:

The value of pi to 7 digits is 3.1415978

Example 1:

```
int digits;
float pi;
scanf( "The value of pi to %d decimal digits is %f",
      &digits, &pi );
```

The value 7 is loaded into *digits* and 3.1415978 is assigned to *pi*. The string literals are matched and ignored. They serve only to move the file position indicator so that the numeric data can be read.

Example 2:

```
short digits;
double pi;
char str[80];
scanf( "%19c %hd %*19c %51f", str, &digits, &tpi );
```

In this example, the number of digits and the value of *pi* are assigned to **short** and **double** variables, respectively. The field width designation in *%51f* causes *scanf()* to read only the first 5 characters of *pi* (3.1415).

The first part of the text is assigned to the array *str[]*; the second part has assignment suppressed by the asterisk. Note that there are only three data arguments even though there are four conversion specifiers because one of them is suppressed. Also note that the data argument for the text string is simply *str*, because an array name by itself is automatically converted to a pointer to the initial element of the array.

Example 3:

```
long digits;
long double pi;
str[80];
scanf( "%*s %*s %*s %*s %*s %ld %*s %*s %*s %Lf",
      &digits, &pi );
```

The number of digits and the value of *pi* are assigned to a **long** and **long double**, respectively. Each word in the input text is read by a *%s* conversion specifier, but assignment is suppressed.

A.13.32 The *setbuf()* Function

```
#include <stdio.h>
void setbuf( FILE *stream, char *buf );
```

The *setbuf()* function is used to change the buffering properties of a stream. Normally, input and output are stored in blocks until the block is filled, and then the entire block is sent to its destination. The size of a block is implementation-defined but is typically 512 or 1024 bytes. This function enables you to make the stream unbuffered. When a stream is unbuffered, characters are sent to their destination immediately. Use *setvbuf()* to change the size of the buffer.

The *setbuf()* function should be called only after a stream has been opened and before it has been read from or written to. Once you have performed an I/O operation on a stream, you cannot change its buffer properties.

To change the default size of a block, you must allocate your own buffer by declaring an array of **chars** of the desired block size. Then pass a pointer to this array as the second argument. Note that this array must exist at least as long as the stream is open. If it has automatic duration, therefore, make sure that its scope is wide enough so that it is not deallocated before the stream is closed. To make a stream unbuffered, pass a null pointer.

The maximum size of a buffer is implementation-defined and is recorded in the constant *BUFSIZ*.

The standard output stream *stdout* is automatically buffered only if the stream does not point to a terminal.

The standard diagnostic stream *stderr* is unbuffered by default.

Except that it returns no value, the *setbuf()* function is equivalent to *setvbuf()* invoked with the values of *_IOFBF* for *mode* and *BUFSIZ* for *size* or (if *buf* is a null pointer) with the value *_IONBF* for *mode*.

A.13.33 The *setvbuf()* Function

```
#include <stdio.h>
int setvbuf( FILE *stream, char *buf, int mode,
             size_t size );
```

The *setvbuf()* function enables you to change the default buffering parameters for a stream. Use *setvbuf()* after you have opened a stream but before you have read from or written to it.

There are three choices for the argument *mode*, each of which is a macro defined in *stdio.h*:

_IOFBF forces I/O to be fully buffered.

_IOLBF causes output to be line buffered.

_IONBF causes I/O to be unbuffered.

If *buf* is not a null pointer, the array it points to may be used as the buffer instead of an array automatically allocated by the runtime system. Note, however, that the array pointed to by *buf* must have a lifetime at least as long as the stream to which it is associated.

The argument *size* specifies the size of the array pointed to by *buf*. The contents of this array at any time are indeterminate.

The *setvbuf()* function returns zero if it is successful. It returns a nonzero value if the arguments are invalid or if the request cannot be honored for some other reason.

A.13.34 The *sprintf()* Function

```
#include <stdio.h>
int sprintf( char *s, const char *format, ... );
```

The *sprintf()* function behaves exactly like *fprintf()*, except that the data are written to a character array instead of an output stream. *sprintf(f)* appends a null character after the last character written. It returns the number of characters assigned, not including the terminating null. See the description of *printf()* for more information.

The *sprintf()* function subsumes the older *ecvt()*, *fcvt()*, and *gcvt()* functions.

A.13.35 The *sscanf()* Function

```
#include <stdio.h>
int sscanf( char *s, const char *format, ... );
```

The *sscanf()* function is the same as *fscanf()*, except that the first argument identifies an array rather than a stream from which to read input. See the description of *scanf()* for more information.

A.13.36 The *tmpfile()* Function

```
#include <stdio.h>
FILE *tmpfile( void );
```

The *tmpfile()* function creates a temporary binary file. The file is opened with update status in binary mode (**wb+**). It is automatically deleted when it is closed, whether explicitly or implicitly. *tmpfile()* returns a pointer to the stream of the new file. If for some reason the file cannot be created, *tmpfile()* returns a null pointer.

A.13.37 The *tmpnam()* Function

```
#include <stdio.h>
char *tmpnam( char *s );
```

Like the *tmpfile()* function, *tmpnam()* is used to create a temporary file. However, *tmpnam()* is more flexible than *tmpfile()*. The *tmpnam()* function enables you to open a file in either binary or text mode, and the file is not automatically deleted.

tmpnam() generates a filename that is guaranteed not to conflict with other filenames. If you pass a null pointer, *tmpnam()* generates a file name but leaves it in an internal static object and returns a pointer to that object. Subsequent calls to *tmpnam()* can modify the file name. If you pass a pointer with a nonzero value, however, *tmpnam()* assumes that you have allocated enough storage for the new name so it generates a name, stores it at the passed address, and returns the pointer argument as the result. The maximum file name length is stored in *L_tmpnam*, which is defined in *<limits.h>*.

The *tmpnam()* function is guaranteed to generate at least 25 unique names before it begins duplicating itself. The actual implementation-defined number of unique names is represented by the constant *TMP_MAX*. The file that is created has the same properties as other files created within the C context. You can open and close it with calls to *fopen()* and *fclose()*. To delete it, you must explicitly *remove()* it.

A.13.38 The *ungetc()* Function

```
#include <stdio.h>
int ungetc( int c, FILE *stream );
```

The *ungetc()* function pushes a character (specified by *c*) back onto the specified input stream. The pushed character will be the next character read assuming there is no intervening *fseek()*. Note that *ungetc()* affects the buffer, but not the file or device associated with the stream. Moreover, *ungetc()* affects the file position indicator in undefined ways, so it is not wise to mix calls to *ungetc()* with calls to *fseek()* that use the *SEEK_CUR* mode.

If *ungetc()* cannot push the character onto a stream, it returns *EOF*. Otherwise, it returns *c*.

A.13.39 The *vfprintf()* Function

```
#include <stdarg.h>
#include <stdio.h>
int vfprintf( FILE *stream, const char *format,
             va_list arg );
```

The *vfprintf()* function can be used in conjunction with the variable argument macros to perform the same operation as *fprintf()*. The difference is that the variable argument list is replaced by the array *arg*, which must be initialized by the *va_start* macro. See Section A.12 on variable argument macros for more information.

A.13.40 The *vprintf()* Function

```
#include <stdarg.h>
#include <stdio.h>
int vprintf( const char *format, va_list arg );
```

The *vprintf()* function can be used in conjunction with the *printf()*. The difference is that the variable argument list is replaced by the array *arg*, which must be initialized by the *va_start* macro. See Section A.12 on variable argument macros for more information.

A.13.41 The *vsprintf()* Function

```
#include <stdarg.h>
#include <stdio.h>
int vsprintf( FILE *stream, const char *format,
             va_list arg );
```

The *vsprintf()* function can be used in conjunction with the variable argument macros to perform the same operation as *sprintf()*. The difference is that the variable argument list is replaced by the array *arg*, which must be initialized by the *va_start* macro. See Section A.12 on variable argument macros for more information.

A.14 General Utilities

The *<stdlib.h>* header file declares four types and a number of functions that fall under the category of "general utilities." This group of functions can be further divided into the following subgroups:

- String Conversion Functions
- Pseudo-Random Number Generation Functions
- Memory Management Functions

- Environment Functions
- Searching and Sorting Functions
- Integer Arithmetic Functions

The types defined in *<stdlib.h>* are

<i>div_t</i>	A structure returned by the <i>div()</i> function.
<i>ldiv_t</i>	A structure returned by the <i>ldiv()</i> function.
<i>size_t</i>	The data type that results from a sizeof expression.
<i>wchar_t</i>	An integral type whose range can represent all characters in the largest character set supported by the compiler.

The *<stdlib.h>* header file also defines several macros:

<i>EXIT_FAILURE</i>	An integral expression that can be returned by the <i>exit()</i> function to indicate unsuccessful termination.
<i>EXIT_SUCCESS</i>	An integral expression that can be returned by the <i>exit()</i> function to indicate successful termination.
<i>MB_CUR_MAX</i>	A positive integer expression that represents the greatest number of bytes that can be used to represent a multibyte character in the current locale.
<i>RAND_MAX</i>	Expands to an integral constant expression whose value represents the maximum value returned by the <i>rand()</i> function.

A.14.1 String Conversion Functions

The following functions convert a string of characters into a numeric value. For example,

```
atoi( "1234" )
```

returns the integer value 1234.

A.14.1.1 The *atof()* Function

```
#include <stdlib.h>
double atof( const char *nptr );
```

The *atof()* function converts the string pointed to by *nptr* into a **double** value. It is equivalent to the *strtod()* function except that it does not have the same error-reporting facilities.

A.14.1.2 The *atoi()* Function

```
#include <stdlib.h>
int atoi( const char *nptr );
```

The *atoi()* function converts the string pointed to by *nptr* into its **int** representation.

A.14.1.3 The *atol()* Function

```
#include <stdlib.h>
long atol( const char *nptr );
```

The *atol()* function converts the string pointed to by *nptr* into its **long int** representation. It is equivalent to the *strtol()* function except that it does not include the same error-reporting facilities.

A.14.1.4 The *strtod()* Function

```
#include <stdlib.h>
double strtod( const char *nptr, char **endptr );
```

The *strtod()* function interprets the string pointed to by *nptr* as a floating-point value and returns its **double** representation. The string may contain leading spaces, which are ignored, followed by an optional plus or minus sign, followed by the floating-point number in either regular or scientific notation. If the string represents an integer value (i.e., there is no decimal point), the *strtod()* function assumes a decimal point following the last digit. If an inappropriate character appears before the first digit following an e or E, the exponent is assumed to be zero.

The function continues reading and processing characters in the string until it reaches a character that cannot be part of the floating-point value. At this point, the function concludes and assigns a pointer to the unrecognized character to *endptr* (if *endptr* is a null pointer, however, no assignment takes place).

Assuming successful completion, the *strtod()* function returns the **double** value of the string. If the function cannot decipher a floating-point value, it returns zero and sets *errno* to *EDOM*. It also assigns the value of *nptr* to the object pointed to by *endptr*, assuming *endptr* is not a null pointer.

If *strtod()* successfully interprets the floating-point value, but the value is too large to fit in a **double**, the function returns *HUGE_VAL* (or negative *HUGE_VAL* if the floating-point value is negative) and sets *errno* to *ERANGE*. If the floating-point value is too small to be represented in a **double**, the function returns zero and sets *errno* to *ERANGE*.

A.14.1.5 The *strtol()* Function

```
#include <stdlib.h>
long strtol( const char *nptr, char **endptr,
             int base );
```

The *strtol()* function converts the string pointed to by *nptr* into its **long int** representation in any base from 2 through 36. Leading white space is ignored and an optional plus or minus sign is allowed.

If the *base* value is zero, then the string is interpreted as a decimal integer constant, possibly preceded by a plus or minus sign but not including an integer suffix. Otherwise, the value of *base* should be between 2 and 36 to indicate the base to be used for conversion. Bases greater than 9 use alphabetic letters from *a* (valued at 10) to *z* (valued at 35). If the value of *base* is 16, the integer may include a *0X* or *0x* prefix to indicate a hexadecimal constant.

The *strtol()* function continues reading characters until it reaches a character that cannot be part of the number. A pointer to this character is assigned to *endptr*. (If *endptr* is a null pointer, however, no assignment takes place.)

Upon successful completion, *strtol()* returns the converted value. If it cannot decipher an integer from the string, it returns zero and sets *errno* to *EDOM*. It also assigns the value of *nptr* to the object pointed to by *endptr*, assuming *endptr* is not a null pointer.

If *strtol()* successfully interprets the integer value but it is too large to fit in a **long int**, the function returns *LONG_MAX* or *LONG_MIN*, depending on the sign of the value, and sets *errno* to *ERANGE*.

A.14.1.6 The *strtoul()* Function

```
#include <stdlib.h>
unsigned long int strtoul( const char *nptr,
                          char **endptr, int base );
```

The *strtoul()* function converts the string pointed to by *nptr* into its **unsigned long int** representation in any base from 2 through 36. Leading white space is ignored. An optional plus or minus sign is *not* allowed.

If the *base* value is zero, then the string is interpreted as a decimal integer constant, not including an integer suffix. Otherwise, the *base* value should be between 2 and 36 to indicate the base to be used for conversion. Bases greater than 9 use alphabetic letters from *a* (valued at 10) to *z* (valued at 35). If the value of *base* is 16, the integer may include a *0X* or *0x* prefix to indicate a hexadecimal constant.

The *strtol()* function continues reading characters until it reaches a character that cannot be part of the number. A pointer to this character is assigned to *endptr*. (If *endptr* is a null pointer, however, no assignment takes place.)

Upon successful completion, *strtol()* returns the converted value. If it cannot decipher an integer from the string, it returns zero and sets *errno* to *EDOM*. It also assigns the value of *nptr* to the object pointed to by *endptr*, assuming *endptr* is not a null pointer.

If *strtol()* successfully interprets the integer value but it is too large to fit in a **unsigned long int**, the function returns *ULONG_MAX* and sets *errno* to *ERANGE*.

A.14.2 Pseudo-Random Number Generator Functions

The *rand()* and *srand()* functions enable you to generate pseudo-random numbers.

A.14.2.1 The *rand()* Function

```
#include <stdlib.h>
int rand( void );
```

The *rand()* function returns an integer in the range 0 through *RAND_MAX*. Successive calls to *rand()* should produce different integers. However, the sequence of random numbers could be the same for each program execution unless you use a different seed value via the *srand()* function.

A.14.2.2 The *srand()* Function

```
#include <stdlib.h>
void srand( unsigned int seed );
```

The *srand()* function uses the argument as a seed for a new sequence of pseudo-random numbers to be returned by subsequent calls to *rand()*. If *srand()* is invoked with the same seed value, the sequence of generated numbers will be the same. The default seed value is 1.

A.14.3 Memory Management Functions

The memory management functions enable you to allocate and deallocate memory dynamically. See Chapter 8 for more information about these functions.

A.14.3.1 The `calloc()` Function

```
#include <stdlib.h>
void *calloc( size_t nmemb, size_t size );
```

The `calloc()` function allocates contiguous space for *nmemb* objects, each of which has a length in bytes specified by *size*. All bits in the allocated space are initialized to zero. `calloc()` returns a pointer to the first byte of the allocated space. If the space cannot be allocated, or if *nelem* or *size* is zero, `calloc()` returns a null pointer.

A.14.3.2 The `free()` Function

```
#include <stdlib.h>
void free( void *ptr );
```

The `free()` function deallocates the space pointed to by *ptr*, which should hold an address returned by a previous call to `calloc()`, `malloc()`, or `realloc()`. If *ptr* is a null pointer, `free()` takes no action. If *ptr* points to an area that was not previously allocated by one of the memory-management functions, or to an area that has already been deallocated, the behavior is undefined. Once a memory area has been freed, assume that its contents have been destroyed. Do not attempt to use the area again. The operating system may recycle the area for future use, but this is beyond user control.

Note that the ANSI Standard does not support `cfree()`, which in many implementations is used to free space allocated by `calloc()`.

A.14.3.3 The `malloc()` Function

```
#include <stdlib.h>
void *malloc( size_t size );
```

The `malloc()` function allocates space for an object whose length is specified by *size*. `malloc()` returns a pointer to the first byte of the allocated space. If the space cannot be allocated, or if *size* is zero, `malloc()` returns a null pointer. The space allocated by `malloc()` is not initialized to any special value.

A.14.3.4 The `realloc()` Function

```
#include <stdlib.h>
void *realloc( void *ptr, size_t size );
```

The `realloc()` function changes the size of a previously allocated space. The *ptr* argument should hold the address of an area previously allocated by `malloc()`, `calloc()`, or `realloc()`. The *size* argument specifies the new size. If the new size is smaller than the old size, the unused portion at the end is

discarded. If the new size is larger than the old size, then all of the old contents are preserved and new memory is tacked on to the end. The new space is not initialized.

realloc() returns a pointer to the first byte of the new object. If the space cannot be allocated, *realloc()* returns a null pointer but leaves the memory area unchanged. If *ptr* is a null pointer, *realloc()* behaves just like a *malloc()* function. If size equals zero, *realloc()* returns a null pointer and frees up the space pointed to by *ptr*. If *ptr* does not point to a previously allocated area, the behavior is undefined.

A.14.4 Environment Functions

The C library contains several functions for communicating with the computer environment, usually through the operating system. These functions enable you to exit prematurely from a program, to specify behavior after program termination, and to execute operating system commands.

A.14.4.1 The *abort()* Function

```
#include <stdlib.h>
void abort( void );
```

The *abort()* function causes abnormal termination of a program. There is no guarantee that buffers will be flushed, that open streams will be closed, or that temporary files will be deleted. The *abort()* function can be turned off by catching the *SIGABRT* signal with the *signal()* function. If the *SIGABRT* signal is not caught, the *abort()* function causes an unsuccessful termination status to be returned to the host environment by means of the function call

```
raise( SIGABRT )
```

If the *SIGABRT* signal is being ignored, *abort()* returns no value. Otherwise, *abort()* causes program termination, so it cannot return to its caller.

A.14.4.2 The *atexit()* Function

```
#include <stdlib.h>
int atexit( void (*func) ( void ) );
```

The *atexit()* function provides a program with a convenient way to clean up an environment before the program exits. The *atexit()* function takes a pointer to a function as an argument and registers that function to be called at program termination. You can register at least 32 functions that will be invoked in the reverse order from which they are registered. The registered functions may not themselves take arguments. When the regis-

tered functions are executed, the program environment is the same as when the *main()* function is called at program start-up. Therefore, these functions should not use variables declared in other modules, even if they have fixed duration.

If it succeeds, *atexit()* returns zero. Otherwise, it returns a nonzero value.

A.14.4.3 The *exit()* Function

```
#include <stdlib.h>
void exit( int status );
```

The *exit()* function produces normal program termination. First, all functions registered by the *atexit()* function are called, in reverse order of their registration. Next, all open output streams are flushed, all open streams are closed, and all files created by the *tmpfile()* function are deleted. Finally, control is returned to the host environment. If the value of status is zero or *EXIT_SUCCESS*, the status returned is *successful termination*. If the returned value is *EXIT_FAILURE*, an implementation-defined meaning of *unsuccessful termination* is indicated. Otherwise the returned status is *unsuccessful termination*. Invoking *exit()* is the same as returning from *main()*, with the exception that the *exit()* call causes all functions registered by *atexit()* to be invoked.

A.14.4.4 The *getenv()* Function

```
#include <stdlib.h>
char *getenv( const char *name );
```

Each environment has an implementation-defined *environment list*, of which each entry has the form *name == value*. The *getenv()* function matches the argument string to one of the names in the list and returns the corresponding *value*. If the argument does not match any names in the list, a null pointer is returned.

A.14.4.5 The *system()* Function

```
#include <stdlib.h>
int system( const char *string );
```

The *system()* function passes the string pointed to by *string* to the host environment to be executed. The string should be a command meaningful to the command processor in the host environment. Before calling *system()*, close all open files because the operating system may access them in unexpected ways.

If *string* is a null pointer, the function call is interpreted as a request to see whether a command processor exists. *system()* returns zero if there is no command processor or a nonzero value to indicate that a command processor exists. If the argument is not a null pointer, *system()* returns an implementation-defined value.

A.14.5 Searching and Sorting Functions

These functions are efficient routines that enable searching for an object in an array and sorting an array. Although they are general-purpose routines, they have usually been finely tuned to run efficiently.

A.14.5.1 The *bsearch()* Function

```
#include <stdlib.h>
void bsearch( const void *key, const void *base,
              size_t nel, size_t *keysize,
              int (*compar) ( const void *,
                              const void * ) );
```

The *bsearch()* function searches an array for an element that matches the object pointed to by *key*. The array itself is identified by the *base* argument, which points to the array's initial element. The *nel* argument specifies the number of array elements to search through, and *keysize* represents the size of each element.

The array must have been previously sorted in ascending order according to a comparison function pointed to by *compar*. The comparison function, which you must supply, takes two arguments and returns a negative number if the object pointed to by the first argument is less than the object pointed to by the second, zero if the two arguments are equal, or a positive number if the first argument is greater than the second. The runtime library supplies a standard comparison function called *memcmp()*.

bsearch() returns a pointer to the matching object in the array or a null pointer if no match is found. If two members compare as equal, a pointer to either one may be returned, depending on the implementation.

A.14.5.2 The *qsort()* Function

```
#include <stdlib.h>
void qsort( void *base, size_t nel, size_t keysize,
            int (*compar)( const void *,
                           const void * ) );
```

The *qsort()* function sorts an array of *nel* objects in ascending order. The initial element of the array is pointed to by *base*, and *keysize* specifies the length of each object. The array is sorted according to a comparison function pointed to by *compar*.

The comparison function, which must be supplied, takes two arguments and returns a negative number if the first argument is less than the second, zero if the two arguments are equal, or a positive number if the first argument is greater than the second. The runtime library supplies a standard comparison function called *memcmp()*. If two elements in the array are equal, their order is unspecified. *qsort()* does not return a value.

A.14.6 Integer Arithmetic Functions

The following functions take integer arguments.

A.14.6.1 The *abs()* Function

```
#include <stdlib.h>
int abs ( int i );
```

The *abs()* function returns the absolute value of *i*. If the result cannot be represented by an **int**, the behavior is undefined. For example, in two's complement notation, the absolute value of the largest negative number cannot be represented.

A.14.6.2 The *div()* Function

```
#include <stdlib.h>
div_t idiv( int numer, int denom );
```

The *div()* function divides *denom* into *numer* and returns a structure containing the quotient and remainder. The structure contains the following members:

```
int quot; /* quotient */
int rem; /* remainder */
```

If the result cannot be represented, the behavior is undefined.

A.14.6.3 The *labs()* Function

```
#include <stdlib.h>
long int labs( long int j );
```

The *labs()* function is equivalent to the *abs()* function, except that the argument and return value have type **long int**.

A.14.6.4 The *ldiv()* Function

```
#include <stdlib.h>
ldiv_t ldiv( long numer, long denom );
```

The *ldiv()* function is identical to *div()*, except that the arguments and results are **long ints** instead of **ints**.

A.15 String-Handling Functions

The C library contains a number of useful functions for manipulating character strings. All of these functions require that the header file *<string.h>* be included. These functions fall into three general categories:

- Functions that begin with *str* operate on null-terminated strings.
- Functions that begin with *strn* operate on strings with a specified maximum length.
- Functions that begin with *mem* operate on arrays of data objects of specified length.

The type *size_t* used by many of these functions is a type defined in *string.h*, which is the type returned by a **sizeof** expression.

A.15.1 The *memchr()* Function

```
#include <string.h>
void *memchr( const void *s, int c, size_t n );
```

The *memchr()* function locates the first occurrence of *c* (converted to an **unsigned char**) in the array pointed to by *s*. If it finds the value, *memchr()* returns a pointer to it; otherwise, *memchr()* returns a null pointer.

A.15.2 The *memcmp()* Function

```
#include <string.h>
int memcmp( const void *s1, const void *s2,
            size_t n );
```

The *memcmp()* function compares the first *n* characters of *s1* with *s2*. Each element in *s1* is compared in turn to the corresponding element in *s2*. As soon as they differ, *memcmp()* determines which is numerically greater. If *s1* is greater, *memcmp()* returns a positive value; if *s2* is greater, a negative value is returned; if the two are equal up to *n* elements, *memcmp()* returns zero. If there are fewer than *n* elements in either array, the results are undefined.

Although the arguments to this function are defined as `void *`, the function was really intended to compare character strings. It may not work as expected for other types of objects. This is especially true for structures that contain holes and objects that have the high-order bit set. See the description of *strcmp()* for contrast.

A.15.3 The *memcpy()* Function

```
#include <string.h>
void *memcpy( void *s1, const void *s2, size_t n );
```

The *memcpy()* function copies *n* characters from string *s2* to string *s1*. If the strings overlap, the behavior is undefined. *memcpy()* returns the value of *s1*.

A.15.4 The *memmove()* Function

```
#include <string.h>
void memmove( void *s1, const void *s2, size_t n );
```

The *memmove()* function copies *n* characters from object *s2* to object *s1*. It is essentially the same as the *memcpy()* function except that it works even when the two objects (*s1* and *s2*) overlap. The *memmove()* function acts as if the *s2* object were first copied to a temporary array and then copied from the temporary holding area to *s1*.

A.15.5 The *memset()* Function

```
#include <string.h>
void *memset( void *s, int c, size_t n );
```

The *memset()* function provides a means for initializing an array to a particular value. It copies the value *c* (converted to an **unsigned char**) into the first *n* elements of array *s*. The *memset()* function returns the value *s*.

A.15.6 The *strcat()* Function

```
#include <string.h>
char *strcat( char *s1, const char *s2 );
```

The *strcat()* function appends a copy of string *s2* to string *s1*. The terminating null character in *s1* is overwritten by the initial character in *s2*. Characters are copied from *s2* until a terminating null character is reached (the null character is also copied). The results are undefined if the two strings overlap. In particular, you cannot necessarily double a string by using the same string as both arguments. *strcat()* returns the value of *s1*.

A.15.7 The *strchr()* Function

```
#include <string.h>
char *strchr( const char *s, int c );
```

The *strchr()* function locates the first occurrence of *c* (converted to a **char**) in the string *s*. The terminating null character is considered part of the string. If the character is located, *strchr()* returns a pointer to it. Otherwise, it returns a null pointer.

A.15.8 The *strcmp()* Function

```
#include <string.h>
int strcmp( const char *s1, const char *s2 );
```

The *strcmp()* function compares string *s1* with string *s2*. If *s1* is less than *s2*, *strcmp()* returns an integer greater than zero; if *s1* is less than *s2*, a negative integer is returned; and if the two strings are equal, *strcmp()* returns zero. See the description of *memcmp()* for contrast.

A.15.9 The *strcoll()* Function

```
#include <string.h>
size_t strcoll( char *to, size_t maxsize,
               const char *from );
```

The *strcoll()* function transforms the string pointed to by *from* so that it is suitable as an argument to *memcmp()* or *strcmp()*. This is particularly applicable to implementations for which the local language forces text to be stored in an inconsistent manner. For example, some languages contain so many characters that they cannot all be stored in a **char**. The *strcoll()* function makes it possible for two strings in such an implementation to be compared to one another.

The transformed string is placed in the array pointed at by *to*. The resulting string will never be more than twice the length of the original string (plus room for the terminating null character). Even fewer characters can be specified to be stored in the *to* array with the *maxsize* argument. *maxsize* represents the maximum number of characters to be placed in the resulting string, including the terminating null character.

If the resulting string contains no more than *maxsize* characters, *strcoll()* returns the number of characters placed in the string. Otherwise, it returns zero and the contents of the *to* array are indeterminate.

A.15.10 The *strcpy()* Function

```
#include <string.h>
char *strcpy( char *s1, const char *s2 );
```

The *strcpy()* function copies the contents of string *s2* into the array pointed to by *s1*. The string identified by *s2* must have a terminating null character, which is also copied. If the string and the array overlap, the results are undefined. *strcpy()* returns the value of *s1*. See the descriptions of *memcpy()* and *strncpy()* for contrast.

A.15.11 The *strcspn()* Function

```
#include <string.h>
size_t strcspn( const char *s1, const char *s2 );
```

Starting from the beginning of *s1*, the *strcspn()* function counts characters that are not present in *s2*. As soon as it matches a character in the two strings, or it reaches the end of *s1*, it returns the number of characters read. The terminating null character is not considered part of *s2*.

A.15.12 The *strerror()* Function

```
#include <string.h>
char *strerror( int errnum );
```

The *strerror()* function returns a pointer to an error message represented by *errnum*. The array that holds the message cannot be modified, but it can be overwritten by subsequent calls to *strerror()*.

A.15.13 The *strlen()* Function

```
#include <string.h>
size_t strlen( const char *s );
```

The *strlen()* function returns the length of the string (number of bytes) pointed to by *s*. The terminating null character is not included in the length.

A.15.14 The *strncat()* Function

```
#include <string.h>
char *strncat( char *s1, const char *s2,
               size_t n );
```

The *strncat()* function appends up to *n* characters from string *s2* to the end of string *s1*. The terminating null character in *s1* is overwritten by the ini-

tial character in *s2*. If the terminating null character in *s2* is reached before *n* characters have been written, the null character is copied, but no other characters are written. If *n* characters are written before a terminating null is encountered, the *strncat()* function appends its own terminating null character to *s1*, so that *n+1* characters are written. The results are undefined if the two strings overlap in memory. *strncat()* returns *s1*.

A.15.15 The *strncmp()* Function

```
#include <string.h>
int strncmp( const char *s1, const char *s2,
             size_t n );
```

The *strncmp()* function is the same as *strcmp()* except that it does not compare more than *n* characters. If *s1* is greater than *s2*, *strncmp()* returns an integer greater than zero; if *s1* is less than *s2*, a negative integer is returned; and if the two strings are equal, *strncmp()* returns zero.

A.15.16 The *strncpy()* Function

```
#include <string.h>
char *strncpy( char *s1, const char *s2,
              size_t n );
```

The *strncpy()* function copies up to *n* characters from the string *s2* into the array pointed to by *s1*. If the string to be copied is shorter than *n* characters, null characters are appended to the array as padding until *n* characters have been written. Note that if the string to be copied is longer than *n* characters, the array that gets the copy will not be null-terminated. If the string and the array overlap, the results are undefined. *strncpy()* returns the value of *s1*. See the descriptions of *memcpy()* and *strcpy()* for contrast.

A.15.17 The *strpbrk()* Function

```
#include <string.h>
char *strpbrk( const char *s1, const char *s2 );
```

The *strpbrk()* function is the inverse of the *strcspn()* function. It locates the first character in *s1* that is also present in *s2*. It returns a pointer to this character, or a null character if no match occurs. The terminating null characters are not included.

A.15.18 The *strrchr()* Function

```
#include <string.h>
char *strrchr( const char *s, int c );
```

The *strrchr()* function locates the last occurrence of *c* (converted to a char) in string *s*. It returns a pointer to this character, or a null pointer if the character is not present in the string.

A.15.19 The *strspn()* Function

```
#include <string.h>
size_t strspn( const char *s1, const char *s2 );
```

The *strspn()* function counts characters in *s1*, starting from the beginning of the string, until it reaches a character that is not present in *s2*. It returns the number of characters counted.

A.15.20 The *strstr()* Function

```
#include <string.h>
char *strstr( const char *s1, const char *s2 );
```

The *strstr()* function locates the first occurrence of string *s2* (not including the terminating null character) in string *s1*. It returns a pointer to the located string in *s1*, or a null pointer if no match occurs.

A.15.21 The *strtok()* Function

```
#include <string.h>
char *strtok( char *s1, const char *s2 );
```

The *strtok()* function divides a string into a number of tokens. The semantics of *strtok()* are somewhat complex. String *s1* is the string to be tokenized, and *s2* contains the separator characters. The *strtok()* function is designed to be called multiple times to fully tokenize *s1*. Its behavior on the first call is somewhat different from its behavior on subsequent calls. The first call to *strtok()* operates as follows:

1. *strtok()* locates the first character in *s1* that is not contained in *s2*. If no such character is found, *strtok()* returns a null pointer. If such a character is found, it represents the beginning of the first token. Ultimately a pointer to this character is returned, but first *strtok()* finds the end of the token, as described in Step 2 below.
2. Assuming it finds the beginning of a token, *strtok()* then looks for a character that *is* contained in *s2*. If it cannot find such a character, then the token extends to the end of *s1*, and subsequent

searches for a token will fail. If it does find such a character, it overwrites it with a null character, which terminates the token. The *strtok()* function then saves a pointer to the next character in *s1* for use in subsequent calls.

After the first call, all subsequent calls to *strtok()* should have *NULL* as the first argument. They begin tokenizing where the last *strtok()* function left off and behave as described in Step 2. Program A-6 illustrates the behavior of the *strtok()* function.

The output is

```
a
b*
c-d
/e
```

Program
A-6

```
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
#include <string.h>

int main( void )
{
    static char s[] = "+a+b* (c d)/e";
    char *token;
    token = strtok( s, "+" ); /* token points to
    "a" */
    printf( "%s\n", token );
    /* token points to "b*" */
    token = strtok( NULL, "(" );
    printf( "%s\n", token );
    /* token points to "c d" */
    token = strtok( NULL, "+*/" );
    printf( "%s\n", token );
    /* token points to "/e" */
    token = strtok( NULL, "+" );
    printf( "%s\n", token );
    /* token is a null pointer */
    token = strtok( NULL, "+" );
    exit( 0 );
}
```

A.15.22 The *strxfrm()* Function

```
#include <string.h>
size_t strxfrm( char *s1, const char *s2,
               size_t n );
```

The *strxfrm()* function transforms a string (*s2*) in some implementation-defined manner so that it is suitable as an argument to the *strcmp()* function. The resulting string is placed in the array pointed to by *s1*. *n*

specifies the maximum length of the transformed string (including the terminating null character).

The only requirement on the transformation is that any two strings transformed by *strxfrm()* and compared with *strcmp()* must return the same result as would occur if they were compared with *strcoll()* prior to the transformation. In other words, the transformation may change certain character codes, but the collating sequence must remain the same.

strxfrm() returns the length of the transformed string, not including the terminating null character.

A.16 Multibyte Character Functions

The ANSI standard defines several runtime routines to assist programmers working with extended character sets that utilize multibyte characters. Whether or not multibyte characters are in use and the interpretation of each multibyte character are determined by the value of the *LC_TYPE* macro and the current locale setting (see Section A.8).

A.16.1 Character Functions

Multibyte characters can have different *shift states*, which determine how each multibyte character is interpreted. A programmer can change shift states by forcing the system to read special multibyte characters. If a call to one of these functions changes the shift state, the shift state will remain changed until a subsequent call changes it again. Passing a null pointer to any of the following functions returns the multibyte character system to its initial shift state.

A.16.1.1 The *mblen()* Function

```
#include <stdlib.h>
int mblen( const char *s, size_t n );
```

If *s* is not a null pointer, *mblen()* returns the number of characters composing the multibyte character pointed to by *s*. If *s* points to a null character, *mblen()* returns 0. If *s* does not point to a valid multibyte character, *mblen()* returns -1.

If *s* is a null pointer, *mblen()* sets the multibyte character system to its initial shift state. The function returns 0 if there is only one shift state and a non-zero value if there exists more than one shift state.

A.16.1.2 The *mbtowc()* Function

```
#include <stdlib.h>
int mbtowc( wchar_t *pwc, const char *s,
            size_t n );
```

If *s* is not a null pointer, the *mbtowc()* function converts a multibyte character into an integer code and stores the code at the address pointed to by *pwc*. The *mbtowc()* function will attempt to determine how many bytes compose each multibyte character and will read that many bytes. At no time, however, will it read more than *n* bytes or the number of bytes specified by the *MB_CUR_MAX* macro, whichever is less. The value returned by *mbtowc()* is 0 if *s* points to a null character, or the number of bytes comprising the multibyte character if *mbtowc()* successfully converts it.

If *s* is a null pointer, *mbtowc()* sets the multibyte character system to its initial shift state. The function returns 0 if there is only one shift state and a nonzero value if there exists more than one shift state.

A.16.1.3 The *wctomb()* Function

```
#include <stdlib.h>
int wctomb( char *s, wchar_t wchar );
```

If *s* is not a null pointer, *wctomb()* converts a character code into its multibyte representation and stores the multibyte character in the array pointed to by *s*. *wctomb()* returns the number of characters in the multibyte character. If *wchar* does not correspond to a valid multibyte character, *wctomb()* returns -1.

If *s* is a null pointer, *wctomb()* sets the multibyte character system to its initial shift state. The function returns 0 if there is only one shift state and a nonzero value if there exists more than one shift state.

A.16.2 Multibyte String Functions

A.16.2.1 The *mbstowcs()* Function

```
#include <stdlib.h>
size_t mbstowcs( wchar_t *pwcs, const char *s,
                 size_t n );
```

The *mbstowcs()* function converts a multibyte character string pointed to by *s* into an array of integer codes that correspond to each character. The codes are placed in the array pointed to by *pwcs*. *mbstowcs()* stops reading multibyte characters as soon as it encounters a null character or it reads *n* multibyte characters, whichever comes first.

Each *mbstowcs()* call begins in the initial shift state. Characters that are read may change the shift state for the duration of the function call, but they have no effect on subsequent calls to any other function.

mbstowcs() returns the number of converted characters, not including the null character, if there is one. If *mbstowcs()* encounters an invalid multibyte character, it returns -1.

A.16.2.2 The *wcstombs()* Function

```
#include <stdlib.h>
size_t wcstombs( char *s, const wchar_t *pwcs,
                 size_t n );
```

The *wcstombs()* function converts an array of codes pointed to by *pwcs* into an array of multibyte characters and stores the characters in the array pointed to by *s*. *wcstombs()* stops converting characters as soon as it encounters a null character or after it has filled *n* bytes in the string pointed to by *s*, whichever comes first.

Each *wcstombs()* call begins in the initial shift state. Characters that are read may change the shift state for the duration of the function call, but they have no effect on subsequent calls to any other function.

wcstombs() returns the number of bytes written to *s*, not including the terminating null character, if there is one. If *wcstombs()* encounters an invalid multibyte code, it returns -1.

A.17 Date and Time Functions

The date and time functions enable you to access the system clock and calendar in a variety of ways. All of these functions require inclusion of the header file *<time.h>*. There are three types of time that these functions return:

- *calendar time* represents the current date and time according to the Gregorian calendar;
- *local time* is the calendar time expressed for a specific time zone;
- *daylight savings time* reflects a temporary change in the local time due to daylight savings regulations.

The header file defines one macro and declares three type definitions. The macro is

CLOCKS_PER_SEC

Represents the number per second of the value returned by the *clock()* function.

The type definitions are:

clock_t Arithmetic type capable of representing time.

time_t Arithmetic type capable of representing time.

tm Structure that holds the components of a calendar time (see below).

The *tm* structure contains the following components at least (it may contain additional components):

```
int tm_sec; /* seconds after the minute - [0, 59]
*/
int tm_min; /* minutes after the hour -- [0, 59]
*/
int tm_hour; /* hours since midnight -- [0, 23] */
int tm_mday; /* day of the month -- [1, 31] */
int tm_mon; /* months since January -- [0, 11] */
int tm_year; /* years since 1900 -- [ ] */
int tm_wday; /* days since Sunday -- [0, 6 ] */
int tm_yday; /* days since January 1 -- [0, 365] */
int tm_isdst; /* daylight savings time flag */
```

The value of *tm_isdst* is positive if daylight savings time is in effect, zero if daylight savings time is not in effect, and negative if the information is not available.

A.17.1 The *asctime()* Function

```
#include <time.h>
char *asctime( const struct tm *timeptr );
```

The *asctime()* function converts the time represented by the structure pointed to by *timeptr* into a character string with the following form:

```
Sun Sep 16 01:03:52 1973\n\0
```

asctime() returns a pointer to the generated string. Subsequent calls to *asctime()* or *ctime()* may overwrite this string.

A.17.2 The *clock()* Function

```
#include <time.h>
clock_t clock( void );
```

The *clock()* function returns the amount of processor time used by the program. To get the value in terms of seconds, divide the returned value by the macro *CLOCKS_PER_SEC*. The behavior of the *clock()* function is largely implementation defined. There is no precise definition for determining when the clock should start counting, and an implementation only needs to give its best approximation. If the processor time is not available, the *clock()* function returns -1 cast to the *clock_t* type.

A.17.3 The *ctime()* Function

```
#include <time.h>
char *ctime( const time_t *timer );
```

The *ctime()* function converts the calendar time pointed to by *timer* to local time in the form of a character string. It is equivalent to

```
asctime( localtime( timer ) )
```

A.17.4 The *difftime()* Function

```
#include <time.h>
double difftime( time_t time1, time_t time0 );
```

The *difftime()* function returns the difference *time1* - *time0*, expressed in seconds.

A.17.5 The *gmtime()* Function

```
#include <time.h>
struct tm *gmtime( const time_t *timer );
```

The *gmtime()* function converts the calendar time pointed to by *timer* into a broken-down time, expressed as Greenwich Mean Time (GMT). The *gmtime()* function returns a pointer to a structure containing the time components. If the GMT is not available, *gmtime()* returns a null pointer. Subsequent calls to *gmtime()* or *localtime()* may point to the same static structure *tm*, which is overwritten by each call.

A.17.6 The *localtime()* Function

```
#include <time.h>
struct tm *localtime( const time_t *timer );
```

The *localtime()* function converts the calendar time pointed to by *timer* into a broken-down time, expressed as local time. The *localtime()* function returns a pointer to a structure containing the time components. Subsequent calls to *gmtime()* or *localtime()* may point to the same static structure *tm*, which is overwritten by each call.

A.17.7 The *mktime()* Function

```
#include <time.h>
time_t mktime( struct tm *timeptr );
```

The *mktime()* function converts a broken-down time in a *tm* structure into a calendar time of the same form returned by the *time()* function. The values of *tm_wday* and *tm_yday* are ignored, and the values of the other fields are not restricted to the values shown in the earlier description of *tm*. In addition to returning a calendar time, *mktime()* also sets the fields in the structure pointed to by *timeptr* to appropriate values. This means that if the original values are out of range, *mktime()* forces them into the ranges listed above. *mktime()* also assigns appropriate values to *tm_wday* and *tm_yday*.

If *mktime()* cannot calculate a returnable calendar time, it returns *(time_t)-1*.

Program A-7 shows how you might use the *mktime()* function to write a function that performs some loop for a specified number of minutes.

Note that the *mktime()* function will work even if the expression

```
when.tm_min += x_minutes
```

is greater than 59.

Program
A-7

```
#include <time.h>

void do_for_x_minutes( int x_minutes )
{
    struct tm when;
    time_t now, deadline;
    time( &now );
    when = *localtime( &now );
    when.tm_min += x_minutes;
    deadline = mktime( &when );
    /* Do foo() for x_minutes */
    while (difftime( time( 0 ), deadline ) > 0)
        foo();
}
```

A.17.8 The *strftime()* Function

```
#include <time.h>
size_t strftime( char *s, size_t maxsize,
                 const char *format,
                 const struct tm *timeptr );
```

The *strftime()* function enables you to construct a string containing information from the structure pointed to by *timeptr*. The format of *strftime()* is similar to *printf()*, where the first argument is a format string that can contain text as well as format specifiers. In this case, however, the format

specifiers are replaced with particular data from the *timeptr* structure. No more than *max_size* characters will be placed in the resulting string pointed to by *s*.

Table A-10: Format specifiers for the time functions.

Format Specifier	Meaning
%a	The abbreviated weekday name.
%A	The full weekday name.
%b	The abbreviated month name.
%B	The full month name.
%c	An appropriate date and time representation.
%d	The day of month as a decimal number (01 - 31).
%H	The hour (24-hour clock) as a decimal number (00 - 23).
%I	The hour (12-hour clock) as a decimal number (01 - 12).
%j	The day of the year as a decimal number (001 - 366).
%m	The month as a decimal number (01 - 12).
%M	The minute as a decimal number (00 - 59).
%p	Either AM or PM (or the equivalent in the local language).
%S	The second as a decimal number (00 - 59).
%U	The week number of the year (Sunday being the first day of the week) as a decimal number (00 - 52).
%w	The weekday as a decimal number (0 - 6)—Sunday is 0.
%W	The week number of the year (where Monday is the first day of the week) as a decimal number (00 - 52).
%x	An appropriate date representation.
%X	An appropriate time representation.
%y	The year (last two digits only) as a decimal number (00 - 99).
%Y	The year (all four digits) as a decimal number.
%Z	The time zone name, or no characters if no time zone exists.
%%	%

The format specifiers and what they are replaced by are listed in Table A-10. The exact value and format of each specifier depend on the particular implementation and the values stored in the structure pointed to by *timeptr*.

If the total number of characters resulting from replacements is not more than *maxsize*, *strftime()* returns the number of characters written to the array pointed to by *s* (not including the terminating null character). Otherwise, *strftime()* returns zero and the contents of the *s* array are indeterminate.

A.17.9 The *time()* Function

```
#include <time.h>
time_t time( time_t *timer );
```

The *time()* function returns the implementation's best approximation of the calendar time. The encoding of the value is unspecified. If *timer* is not a null pointer, the calendar time is also assigned to the object that it points to. If the calendar time is unavailable, *time()* returns -1.

B

Implementation Limits

Every C compiler imposes certain limitations upon the types of programs it will compile, such as the maximum length of variable names and the maximum length of lines in source files. These constraints are called *translation limits* because they concern how the compiler translates source text. There are also *numerical limits*, which concern the minimum and maximum values that can be represented by variously typed objects. For both types of limits, the ANSI Standard defines minimum values. An ANSI-conforming C compiler must support at least these minimum values but is free to exceed these limitations. In fact, ANSI recommends that implementations avoid imposing any limits wherever possible. Your compiler documentation should list all limits.

B.1 Translation Limits

An ANSI-conforming compiler must support at least the following:

- 15 nesting levels of compound statements, iteration control structures, and selection control structures
- 6 nesting levels in conditional compilation
- 12 pointer, array, and function declarators modifying a basic type in a declaration
- 32 expressions nested by parentheses

- 31 significant initial characters in an internal identifier or macro name
- 6 significant initial characters in an external identifier
- 511 external identifiers in one source file
- 127 identifiers with block scope in one block
- 1024 macro names simultaneously defined in one source file
- 31 parameters in one function definition or call
- 31 parameters in one macro definition or invocation
- 509 characters in a source line
- 509 characters in a string literal (after concatenation)
- 32767 bytes in an array or structure
- 8 nesting levels for **#included** files
- 257 **case** labels in a **switch** statement
- 127 members in a single structure or union
- 127 enumeration constants in a single enumeration
- 15 levels of nested structure or union definitions in a declaration

B.2 Numerical Limits

The ANSI Standard defines the minimum range of values that each scalar type of object must be able to represent. For integral objects, the low end and high end of these ranges are recorded in macro constants that are defined in the *limits.h* header file. Implementations may, of course, support larger ranges.

For floating-point types, the ANSI Standard defines a number of macros that describe an implementation's floating-point representation. These macros are defined in the *float.h* header file.

B.2.1 Sizes of Integral Types

The following table lists the macro names defined in *limits.h*, their meanings, and their minimum value for ANSI-conforming implementations:

<i>Macro Name</i>	<i>Value</i>	<i>Meaning</i>
CHAR_BIT	8	minimum number of bits for smallest object that is not a bit field (i.e., a byte)
SCHAR_MIN	-127	minimum value for an object of type signed char
SCHAR_MAX	+127	maximum value for an object of type signed char
UCHAR_MAX	255	maximum value for an object of type unsigned char
CHAR_MIN	*	minimum value for an object of type char
CHAR_MAX	*	maximum value for an object of type char
SHRT_MIN	-32767	minimum value for an object of type short int
SHRT_MAX	+32767	maximum value for an object of type short int
USHRT_MAX	65535	maximum value for an object of type unsigned short int
INT_MIN	-32767	minimum value for an object of type int
INT_MAX	+32767	maximum value for an object of type int
UINT_MAX	65535	maximum value for an object of type unsigned int
LONG_MIN	-2147483647	minimum value for an object of type long int
LONG_MAX	+2147483647	maximum value for an object of type long int
ULONG_MAX	4294967295	maximum value for an object of type unsigned long int

* If chars are signed by default, the value of *CHAR_MIN* should be the same as that of *SCHAR_MIN* and the value of *CHAR_MAX* should be the same as that of *SCHAR_MAX*. If chars are unsigned by default, the value of *CHAR_MIN* should be zero, and the value of *CHAR_MAX* should be the same as that of *UCHAR_MAX*.

B.2.2 Characteristics of Floating-Point Types

The ANSI Standard defines the characteristics of floating-point types in terms of a model that describes a representation of floating-point numbers and values that provide information about an implementation's floating-point arithmetic. We recommend that you read the ANSI Standard for a detailed discussion of this model. In this section, we present information about the limits for floating-point objects.

For any floating-point object, there are four limits:

- max* The largest positive value that can be represented.
- min* The largest negative value that can be represented.
- epsilon* The minimum positive number, *x*, such that $1.0 + x \neq 1.0$
- precision* The number of decimal digits of precision.

Each ANSI-conforming compiler provides values for these four limits in names. These names, however, need not be constants—they can also represent expressions evaluated at runtime. The names for each of the floating-point types and the minimum values that an ANSI-conforming compiler must support are shown below. Compilers are free to support values greater in magnitude (absolute value) to those shown, with the same size.

Maximum Value		
float	<i>FLT_MAX</i>	1e+37
double	<i>DBL_MAX</i>	1e+37
long double	<i>LDBL_MAX</i>	1e+37

Minimum Value		
float	<i>FLT_MIN</i>	1e-37
double	<i>DBL_MIN</i>	1e-37
long double	<i>LDBL_MIN</i>	1e-37

Epsilon		
float	<i>FLT_EPSILON</i>	1e-5
double	<i>DBL_EPSILON</i>	1e-9
long double	<i>LDBL_EPSILON</i>	1e-9

<i>Precision</i>		
float	<i>FLT_DIG</i>	6
double	<i>DBL_DIG</i>	10
long double	<i>LDBL_DIG</i>	10

C

Differences Between the ANSI and K&R Standards

This appendix summarizes the differences between the K&R standard and the ANSI Standard. The references listed in each section point to other parts of this book where you can find more information about the topic.

C.1 Source Translation Differences

The differences listed in this section relate to the lexical analysis stage of compilation.

C.1.1 Name Length

ANSI: Compilers must support internal names of at least 31 characters and external names of at least 6 characters.

K&R: Compilers must support internal names of at least 8 characters; external names may be shorter.

Reference: Section 3.4: Names; Section 8.3: Global Variables

C.1.2 Continuation Character

ANSI: The continuation character may be used to carry names as well as string literals to the next line.

K&R: The continuation character may be used only to continue string literals.

Reference: Section 3.8.1: Continuation Character

C.1.3 String Concatenation

ANSI: Supports this new feature, which causes the compiler to concatenate adjacent string literals into a single string.

K&R: Does not support this feature.

Reference: Section 7.10.6: String Concatenation

C.1.4 Trigraph Sequences

ANSI: Supports trigraph sequences for entering characters not available on some keyboards.

K&R: Does not support trigraph sequences.

Reference: Section 4.2.3: Trigraph Sequences

C.1.5 Alert and Vertical Tab Escape Sequences

ANSI: Requires compilers to support `\a` (alert) and `\v` (vertical tab) escape sequences.

K&R: Does not require compilers to support `\a` and `\v`.

Reference: Section 4.2.2: Escape Character Sequences

C.1.6 Hexadecimal Escape Sequences

ANSI: Supports escape sequences of the form:

`\xhhh`

where the *h*'s are hexadecimal digits.

K&R: Does not support hexadecimal escape sequences.

Reference: Section 4.2.2: Escape Character Sequences

C.2 Data Type Differences

The entries in this section cover differences in data types.

C.2.1 signed Type Specifier

ANSI: Supports the new keyword, **signed**, which explicitly makes an integral type signed.

K&R: Does not support the **signed** keyword.

Reference: Section 4.1.4: signed Integers

C.2.2 long double Type

ANSI: Supports **long double** type.

K&R: Does not support **long double** type.

Reference: Section 4.3: Floating-Point Types; Section 4.6.4 Mixing Floating-Point Values

C.2.3 unsigned short, unsigned long, and unsigned char Types

ANSI: Explicitly requires C compilers to support these types.

K&R: Implies that these types are not legal.

Reference: Section 4.1.3: Unsigned Integers; Section 4.10: Typedefs

C.2.4 The void Type

ANSI: Included as a new type, an object of which cannot be used. Pointers to **void** may be converted to pointers of any other type object.

K&R: Not supported at all in the original version.

Reference: Section 4.9: The void Data Type.

C.2.5 Enumeration Types

ANSI: Supported as integer type.

K&R: Not supported in the original document, although added in a later draft.

Reference: Section 4.8: Enumeration Types

C.2.6 Byte Length

ANSI: A byte must be at least 8 bits long.

K&R: Bytes have unspecified size.

Reference: Section 4.1.2: Different Types of Integers

C.2.7 Minimum Ranges for Integral Types

ANSI: Imposes ranges that must be met for each type.

K&R: Does not impose ranges but lists typical sizes.

Reference: Section 4.1.2: Different Types of Integers; Appendix B—Implementation Limits

C.2.8 Unsigned Constants

ANSI: Allows you to append a `u` or `U` to an integral constant to make it unsigned.

K&R: Does not support unsigned constants.

Reference: Section 4.2.1: Unsigned Constants

C.2.9 “8” and “9” Not Allowed in Octal Constants

ANSI: Does not allow the digits 8 and 9 to be used in an octal constant.

K&R: Allows the use of 8 and 9 (which have octal values 10 and 11).

Reference: Section 4.1.2: Different Kinds of Integer Constants

C.2.10 float and long double Constants

ANSI: Allows you to append an `f` or `F` to a floating-point constant to give it type `float`, or an `l` or `L` to give it type `long double`.

K&R: Does not support `float` and `long double` constants.

Reference: Section 4.4.1: float and long double Constants

C.2.11 Type of Integer Constants

ANSI: Has defined rules for determining type.

K&R: Rules for type determination are vague.

Reference: Section 4.2: Different Kinds of Integer Constants

C.2.12 Conversion Rules for Mixing signed and unsigned Types

ANSI: Uses value-preserving rules.

K&R: Uses sign-preserving rules.

Reference: Section 4.6.3: Unsigned Conversions

C.3 Statement Differences

There is only one significant difference concerning control flow statements.

C.3.1 Controlling Expression of a switch Statement

ANSI: Allows the controlling expression of a switch statement to have any integral type.

K&R: States that the controlling expression must have type `int`.

Reference: Section 5.2.1: Syntax of a `switch` Statement

C.4 Expression Differences

This section lists several differences involving the way expressions are evaluated.

C.4.1 Unsigned Conversions

ANSI: Uses value-preserving rules.

K&R: Uses sign-preserving rules.

Reference: Section 4.6.3: Unsigned Conversions

C.4.2 Unary Plus Operator

ANSI: Supports a unary plus operator.

K&R: Does not support a unary plus operator.

Reference: Section 6.2: Unary Plus and Minus Operators

C.4.3 Float Expressions

ANSI: Does not require the compiler to convert all **float** operands to **double**.

K&R: Requires conversion of all **float** operands to **double**.

Reference: Section 4.6.4: Mixing Floating-Point Values

C.4.4 Shifting by a long int and unsigned int

ANSI: States that the type of the shift count does not affect the type of the left-hand operand.

K&R: Implies that shifting by a **long int** or **unsigned int** forces the compiler to convert the left operand to a **long int** or **unsigned int**, respectively.

Reference: Section 6.9.1: Shift Operators

C.4.5 Structure Assignment

ANSI: Allows a structure to be assigned to a structure variable, provided that the two operands share the same structure type.

K&R: Does not support structure assignment.

Reference: Section 9.1.12: Assigning Structures

C.4.6 Passing Structures as Arguments

ANSI: Supports passing structures as function arguments.

K&R: Does not allow structures to be passed as function arguments.

Reference: Section 9.1.10: Passing Structures as Function Arguments

C.4.7 Pointers to Functions

ANSI: Allows you to omit the dereferencing operator when invoking a function through a pointer to a function. If *pf* is a function, then

`pf ();`

is the same as

`(*pf) ();`

K&R: Implies that the dereferencing operator is required.

Reference: Section 10.3.3: Calling a Function Using Pointers

C.5 Storage Class and Initialization Differences

The ANSI Standard includes several extensions to storage classes and initializations.

C.5.1 Function Prototypes

ANSI: Supports a new feature that allows you to declare the number and type of arguments to a function defined elsewhere. Prototyping enables the compiler to perform argument type-checking.

K&R: Not supported.

Reference: Section 10.2.5: Function Prototypes

C.5.2 The `const` Storage-Class Modifier

ANSI: Supports `const`, which indicates that the object's value may not be changed.

K&R: Does not support `const`.

Reference: Section 8.5: The `const` Storage-Class Modifier

C.5.3 The `volatile` Storage-Class Modifier

ANSI: Supports `volatile`, which indicates that the object's value can change in ways that the compiler cannot predict.

K&R: Does not support `volatile`.

Reference: Section 8.6: The `volatile` Storage-Class Modifier

C.5.4 Definitions vs. Allusions

ANSI: Uses the presence or absence of an initializer to determine whether a declaration is a definition or an allusion.

K&R: Uses the presence or absence of the `extern` specifier to determine whether a declaration is a definition or an allusion.

Reference: Section 8.3.1: Definitions and Allusions

C.5.5 Initializing Automatic Arrays and Structures

ANSI: Permitted.

K&R: Not permitted.

Reference: Section 7.3: Initializing Arrays; Section 9.1.1: Initializing Structures

C.5.6 Scope of Function Arguments

ANSI: Arguments declared as function parameters have the same scope as objects declared in the function's top-level block.

K&R: Function arguments may be hidden by declarations of the same name in the top-level block.

Reference: Section 8.2.2: Function Scope

C.5.7 struct and union Name Spaces

ANSI: Each structure and union has its own name space, which means that fields in different structures or unions can have the same name without conflict.

K&R: Places all structure and union fields in the same name space.

Reference: Section 9.1.5: struct and union Name Spaces

C.5.8 Initialization of Union Members

ANSI: Supported—initializes the first member of the union.

K&R: Not supported.

Reference: Section 9.3.2: Initializing Unions

C.6 Preprocessor Differences

The preprocessor defined by the ANSI Standard differs substantially from the preprocessor described by the K&R Standard.

C.6.1 Formatting Preprocessor Lines

ANSI: Requires the pound sign to be the first *nonspace* character on a line.

K&R: Requires the pound sign to be the first character on a line.

Reference: Chapter 11: The C Preprocessor.

C.6.2 Recursive Macro Definitions

ANSI: Prevents infinite recursion of a macro by inhibiting the expansion of a macro name in its own definition.

K&R: Provides no mechanism to avoid infinite recursion of a macro.

Reference: Section 11.1.3: Using a Macro Name in Its Own Definition

C.6.3 Redefining Macro Names

ANSI: Requires an intervening **#undef** of the macro name if the redefinition is different from the current definition.

K&R: Does not require an intervening **#undef**.

Reference: Section 11.1.2: Removing a Macro Definition

C.6.4 String Producer

ANSI: Supports a new preprocessor operator (**#**) that surrounds its argument with quotes when it expands.

K&R: Does not support the string-producing operator.

Reference: Section 11.1.6: String Producer

C.6.5 Built-In Macros

ANSI: Requires implementations to define five built-in macros: **__LINE__**, **__FILE__**, **__TIME__**, **__DATE__**, **__STDC__**.

K&R: Does not require implementations to define any built-in macros (nor does it prohibit them from doing so).

Reference: Section 11.1.5: Built-In Macros

C.6.6 Token Pasting

ANSI: Supports a new preprocessor operator (**##**) that pastes two preprocessor tokens.

K&R: Does not support the paste operator.

Reference: Section 11.1.7: Token Pasting

C.6.7 The **#elif** Directive

ANSI: Supports **#elif**.

K&R: Does not support **#elif**.

Reference: Section 11.2: Conditional Compilation

C.6.8 The `defined` Operator

ANSI: Supports the **`defined`** preprocessor operator.

K&R: Does not support **`defined`**.

Reference: Section 11.2.1: Testing Macro Existence

C.6.9 The `#error` Directive

ANSI: Supports **`#error`**, which enables the output of error messages during the preprocessing stage of compilation.

K&R: Does not support **`#error`**.

Reference: Section 11.4.1: The `#error` Directive

C.6.10 The `#pragma` Directive

ANSI: Supports **`#pragma`**, which allows implementations to add their own preprocessing directives.

K&R: Does not support **`#pragma`**.

Reference: Section 11.4.2: The `#pragma` Directive

D

Reserved Names

The C language, as defined by the ANSI Standard, contains a number of reserved words and names that you should not use as private variable names. The reserved words fall into several categories:

- **Keywords**—You may not use keywords for names of objects.
- **Runtime Function Names**—You should avoid using function names, except when you want to write your own version of a standard function or macro.
- **Macro Names**—The runtime library header files contain definitions for many constant names. You should avoid using these names for variables.
- **Type Names**—Some of the header files define types (with typedefs) that are applied to the arguments or the function return type.
- **Preprocessor Command Names**—In general, the preprocessor names do not create conflicts because they must be preceded by a pound sign, which makes their meaning unambiguous. However, you cannot write something like

```
#define define
```

Table D-1 lists all of these reserved names. For macros and functions, the table also shows where they are defined. Although it is possible to avoid naming conflicts by not including the header file in which a reserved name is defined, this is a dangerous practice because you may need to include the header file at a later date. We recommend that you avoid using these names for private variables.

In addition to the names listed in Table D-1, you should also consider all names beginning with an underscore to be reserved for system use. Fi-

nally, ANSI has reserved all names beginning with two underscores, or an underscore followed by an uppercase letter, for future use.

Table D-1: Reserved Names

__DATE__	macro defined by the implementation
__FILE__	macro defined by the implementation
_IOFBF	macro defined by the implementation
_IOLBF	macro defined by the implementation
_IONBF	macro defined by the implementation
__LINE__	macro defined by the implementation
__STDC__	macro defined by the implementation
__TIME__	macro defined by the implementation
abort	function defined in <i>assert.h</i>
abs	function defined in <i>stdlib.h</i>
acos	function defined in <i>math.h</i>
asctime	function defined in <i>time.h</i>
asin	function defined in <i>math.h</i>
assert	macro defined in <i>assert.h</i>
atan	function defined in <i>math.h</i>
atan2	function defined in <i>math.h</i>
atexit	function defined in <i>stdlib.h</i>
atof	function defined in <i>stdlib.h</i>
atoi	function defined in <i>stdlib.h</i>
atol	function defined in <i>stdlib.h</i>
auto	keyword (storage class specifier)
break	keyword (statement)
bsearch	function defined in <i>stdlib.h</i>
BUFSIZ	macro defined in <i>stdio.h</i>
calloc	function defined in <i>stdlib.h</i>

Table D-1 (continued)

case	keyword (label)
ceil	function defined in <i>math.h</i>
CHAR_BIT	macro defined in <i>limits.h</i>
CHAR_MAX	macro defined in <i>limits.h</i>
CHAR_MIN	macro defined in <i>limits.h</i>
clearerr	function defined in <i>stdio.h</i>
clock	function defined in <i>time.h</i>
clock_t	type defined in <i>time.h</i>
CLOCKS_PER_SEC	macro defined in <i>time.h</i>
const	keyword (storage class modifier)
continue	keyword (statement)
cos	function defined in <i>math.h</i>
cosh	function defined in <i>math.h</i>
ctime	function defined in <i>time.h</i>
DBL_EPSILON	macro defined in <i>float.h</i>
DBL_MANT_DIG	macro defined in <i>float.h</i>
DBL_DIG	macro defined in <i>float.h</i>
DBL_MIN	macro defined in <i>float.h</i>
DBL_MIN_EXP	macro defined in <i>float.h</i>
DBL_MIN_10_EXP	macro defined in <i>float.h</i>
DBL_MAX	macro defined in <i>float.h</i>
DBL_MAX_EXP	macro defined in <i>float.h</i>
DBL_MAX_10_EXP	macro defined in <i>float.h</i>
default	keyword (label)
defined	preprocessing operator
difftime	function defined in <i>time.h</i>
div_t	type defined in <i>stdlib.h</i>
do	keyword (statement)
double	keyword (type specifier)

Table D-1 (continued)

EDOM	macro defined in <i>float.h</i>
else	keyword (statement)
enum	keyword (type specifier)
EOF	macro defined in <i>stdio.h</i>
ERANGE	macro defined in <i>float.h</i>
errno	macro defined in <i>stddef.h</i>
exit	function defined in <i>stdlib.h</i>
exp	function defined in <i>math.h</i>
extern	keyword (storage class specifier)
fabs	function defined in <i>math.h</i>
fclose	function defined in <i>stdio.h</i>
feof	function defined in <i>stdio.h</i>
ferror	function defined in <i>stdio.h</i>
fflush	function defined in <i>stdio.h</i>
fgetc	function defined in <i>stdio.h</i>
fgetpos	function defined in <i>stdio.h</i>
fgets	function defined in <i>stdio.h</i>
FILE	type defined in <i>stdio.h</i>
float	keyword (type specifier)
floor	function defined in <i>math.h</i>
FLT_DIG	macro defined in <i>float.h</i>
FLT_EPSILON	macro defined in <i>float.h</i>
FLT_MANT_DIG	macro defined in <i>float.h</i>
FLT_MAX	macro defined in <i>float.h</i>
FLT_MAX_EXP	macro defined in <i>float.h</i>
FLT_MAX_10_EXP	macro defined in <i>float.h</i>
FLT_MIN	macro defined in <i>float.h</i>
FLT_MIN_EXP	macro defined in <i>float.h</i>
FLT_MIN_10_EXP	macro defined in <i>float.h</i>

Table D-1 (*continued*)

FLT_RADIX	macro defined in <i>float.h</i>
FLT_ROUNDS	macro defined in <i>float.h</i>
fmod	function defined in <i>math.h</i>
fopen	function defined in <i>stdio.h</i>
for	keyword (statement)
fpos_t	type defined in <i>stdio.h</i>
fprintf	function defined in <i>stdio.h</i>
fputc	function defined in <i>stdio.h</i>
fputs	function defined in <i>stdio.h</i>
fread	function defined in <i>stdio.h</i>
free	function defined in <i>stdlib.h</i>
freopen	function defined in <i>stdio.h</i>
frexp	function defined in <i>math.h</i>
fscanf	function defined in <i>stdio.h</i>
fseek	function defined in <i>stdio.h</i>
fsetpos	function defined in <i>stdio.h</i>
ftell	function defined in <i>stdio.h</i>
fwrite	function defined in <i>stdio.h</i>
getc	function defined in <i>stdio.h</i>
getchar	function defined in <i>stdio.h</i>
getenv	function defined in <i>stdlib.h</i>
gets	function defined in <i>stdio.h</i>
gmtime	function defined in <i>stdlib.h</i>
goto	keyword (statement)
HUGE_VAL	macro defined in <i>math.h</i>
if	keyword (statement)
int	keyword (type specifier)
INT_MAX	macro defined in <i>limits.h</i>
INT_MIN	macro defined in <i>limits.h</i>

Table D-1 (continued)

isalnum	function defined in <i>ctype.h</i>
isalpha	function defined in <i>ctype.h</i>
isctrl	function defined in <i>ctype.h</i>
isdigit	function defined in <i>ctype.h</i>
isgraph	function defined in <i>ctype.h</i>
islower	function defined in <i>ctype.h</i>
isprint	function defined in <i>ctype.h</i>
ispunct	function defined in <i>ctype.h</i>
isspace	function defined in <i>ctype.h</i>
isupper	function defined in <i>ctype.h</i>
isxdigit	function defined in <i>ctype.h</i>
jmp_buf	type defined in <i>setjmp.h</i>
L_tmpnam	macro defined in <i>stdio.h</i>
labs	function defined in <i>math.h</i>
LDBL_DIG	macro defined in <i>float.h</i>
LDBL_EPSILON	macro defined in <i>float.h</i>
LDBL_MANT_DIG	macro defined in <i>float.h</i>
LDBL_MAX	macro defined in <i>float.h</i>
LDBL_MAX_EXP	macro defined in <i>float.h</i>
LDBL_MAX_10_EXP	macro defined in <i>float.h</i>
LDBL_MIN	macro defined in <i>float.h</i>
LDBL_MIN_EXP	macro defined in <i>float.h</i>
LDBL_MIN_10_EXP	macro defined in <i>float.h</i>
ldexp	function defined in <i>math.h</i>
ldiv	function defined in <i>stdlib.h</i>
ldiv_t	type defined in <i>stdlib.h</i>
localtime	function defined in <i>time.h</i>
log	function defined in <i>math.h</i>
log10	function defined in <i>math.h</i>

Table D-1 (*continued*)

long	keyword (type specifier)
longjmp	function defined in <i>setjmp.h</i>
LONG_MAX	macro defined in <i>limits.h</i>
LONG_MIN	macro defined in <i>limits.h</i>
main	function defined by the implementation
malloc	function defined in <i>stdlib.h</i>
memchr	function defined in <i>string.h</i>
memcmp	function defined in <i>string.h</i>
memcpy	function defined in <i>string.h</i>
memmove	function defined in <i>string.h</i>
memset	function defined in <i>string.h</i>
mktime	function defined in <i>time.h</i>
modf	function defined in <i>math.h</i>
NDEBUG	macro defined in <i>assert.h</i>
NULL	macro defined in <i>stddef.h</i>
offsetof	macro defined in <i>stddef.h</i>
OPEN_MAX	macro defined in <i>stdio.h</i>
perror	function defined in <i>stdio.h</i>
pow	function defined in <i>math.h</i>
printf	function defined in <i>stdio.h</i>
ptrdiff_t	type defined in <i>stddef.h</i>
putc	function defined in <i>stdio.h</i>
putchar	function defined in <i>stdio.h</i>
puts	function defined in <i>stdio.h</i>
qsort	function defined in <i>stdlib.h</i>
raise	function defined in <i>signal.h</i>
rand	function defined in <i>stdlib.h</i>
RAND_MAX	macro defined in <i>stdlib.h</i>

Table D-1 (*continued*)

realloc	function defined in <i>stdlib.h</i>
register	keyword (storage class specifier)
remove	function defined in <i>stdio.h</i>
rename	function defined in <i>stdio.h</i>
return	keyword (statement)
rewind	function defined in <i>stdio.h</i>
SCHAR_MAX	macro defined in <i>limits.h</i>
SCHAR_MIN	macro defined in <i>limits.h</i>
SEEK_CUR	macro defined in <i>stdio.h</i>
SEEK_END	macro defined in <i>stdio.h</i>
SEEK_SET	macro defined in <i>stdio.h</i>
setbuf	function defined in <i>stdio.h</i>
setjmp	function defined in <i>setjmp.h</i>
setlocale	function defined in <i>locale.h</i>
setvbuf	function defined in <i>stdio.h</i>
short	keyword (type specifier)
SHRT_MAX	macro defined in <i>limits.h</i>
SHRT_MIN	macro defined in <i>limits.h</i>
sig_atomic_t	type defined in <i>signal.h</i>
SIG_DFL	macro defined in <i>signal.h</i>
SIG_ERR	macro defined in <i>signal.h</i>
SIG_IGN	macro defined in <i>signal.h</i>
SIGABRT	macro defined in <i>signal.h</i>
SIGFPE	macro defined in <i>signal.h</i>
SIGILL	macro defined in <i>signal.h</i>
SIGINT	macro defined in <i>signal.h</i>
signal	function defined in <i>signal.h</i>
signed	keyword (type specifier)
SIGSEGV	macro defined in <i>signal.h</i>

Table D-1 (*continued*)

SIGTERM	macro defined in <i>signal.h</i>
sin	function defined in <i>math.h</i>
sinh	function defined in <i>math.h</i>
size_t	type defined in <i>stddef.h</i>
sizeof	keyword (operator)
sprintf	function defined in <i>stdio.h</i>
sqrt	function defined in <i>math.h</i>
srand	function defined in <i>stdlib.h</i>
sscanf	function defined in <i>stdio.h</i>
static	keyword (storage class specifier)
stderr	file pointer defined by the implementation
stdin	file pointer defined by the implementation
stdout	file pointer defined by the implementation
strcat	function defined in <i>string.h</i>
strchr	function defined in <i>string.h</i>
strcmp	function defined in <i>string.h</i>
strcoll	function defined in <i>string.h</i>
strcpy	function defined in <i>string.h</i>
strcspn	function defined in <i>string.h</i>
strerror	function defined in <i>string.h</i>
strftime	function defined in <i>time.h</i>
strlen	function defined in <i>string.h</i>
strncat	function defined in <i>string.h</i>
strncmp	function defined in <i>string.h</i>
strncpy	function defined in <i>string.h</i>
strpbrk	function defined in <i>string.h</i>
strrchr	function defined in <i>string.h</i>

Table D-1 (*continued*)

strspn	function defined in <i>string.h</i>
strstr	function defined in <i>string.h</i>
strtod	function defined in <i>string.h</i>
strtok	function defined in <i>string.h</i>
strtol	function defined in <i>string.h</i>
strtoul	function defined in <i>string.h</i>
switch	keyword (statement)
system	function defined in <i>stdlib.h</i>
tan	function defined in <i>math.h</i>
tanh	function defined in <i>math.h</i>
time	function defined in <i>time.h</i>
time_t	type defined in <i>time.h</i>
tm	type defined in <i>time.h</i>
TMP_MAX	macro defined in <i>stdio.h</i>
tmpfile	function defined in <i>stdio.h</i>
tmpnam	function defined in <i>stdio.h</i>
tolower	function defined in <i>ctype.h</i>
toupper	function defined in <i>ctype.h</i>
typedef	keyword (type specifier)
UCHAR_MAX	macro defined in <i>limits.h</i>
UINT_MAX	macro defined in <i>limits.h</i>
ULONG_MAX	macro defined in <i>limits.h</i>
ungetc	function defined in <i>stdio.h</i>
unsigned	keyword (type specifier)
USHRT_MAX	macro defined in <i>limits.h</i>
va_arg	macro defined in <i>stdarg.h</i>
va_end	function defined in <i>stdarg.h</i>
va_list	type defined in <i>stdarg.h</i>
va_start	macro defined in <i>stdarg.h</i>

Table D-1 (*continued*)

vfprintf	function defined in <i>stdio.h</i>
void	keyword (type specifier)
volatile	keyword (storage class modifier)
vprintf	function defined in <i>stdio.h</i>
vsprintf	function defined in <i>stdio.h</i>
wchar_t	macro defined in <i>stddef.h</i>
while	keyword (statement)

E

ASCII Codes

<i>oct</i>	<i>dec</i>	<i>hex</i>	<i>char</i>
0	0	0	NUL ^@
1	1	1	SOH ^A
2	2	2	STX ^B
3	3	3	ETX ^C
4	4	4	EQT ^D
5	5	5	ENQ ^F
6	6	6	ACK ^F
7	7	7	BEL ^G
10	8	8	BS ^H
11	9	9	TAB ^I
12	10	A	LF ^J
13	11	B	VT ^K
14	12	C	FF ^L
15	13	D	VR ^M
16	14	E	SO ^N
17	15	F	SI ^O
20	16	10	DLE ^P

<i>oct</i>	<i>dec</i>	<i>hex</i>	<i>char</i>
21	17	11	DC1 ^Q
22	18	12	DC2 ^R
23	19	13	DC3 ^S
24	20	14	DC4 ^T
25	21	15	NAK ^U
26	22	16	SYN ^V
27	23	17	ETB ^W
30	24	18	CAN ^X
31	25	19	EM ^Y
32	26	1A	SUB ^Z
33	27	1B	ESC ^[
34	28	1C	FS ^\
35	29	1D	GS ^]
36	30	1E	RS ^^
37	31	1F	US ^_
40	32	20	SPACE
41	33	21	!

<i>oct</i>	<i>dec</i>	<i>hex</i>	<i>char</i>
42	34	22	“
43	35	23	#
44	36	24	\$
45	37	25	%
46	38	26	&
47	39	27	‘
50	40	28	(
51	41	29	)
52	42	2A	*
53	43	2B	+
54	44	2C	,
55	45	2D	-
56	46	2E	.
57	47	2F	/
60	48	30	0
61	49	31	1
62	50	32	2
63	51	33	3
64	52	34	4
65	53	35	5
66	54	36	6
67	55	37	7
70	56	38	8
71	57	39	9
72	58	3A	:
73	59	3B	;
74	60	3C	<
75	61	3D	=
76	62	3E	>
77	63	3F	?
100	64	40	@
101	65	41	A
102	66	42	B
103	67	43	C
104	68	44	D
105	69	45	E
106	70	46	F
107	71	47	G
110	72	48	H
111	73	49	I
112	74	4A	J
113	75	4B	K
114	76	4C	L
115	77	4D	M
116	78	4E	N
117	79	4F	O
120	80	50	P
121	81	51	Q
122	82	52	R
123	83	53	S
124	84	54	T
125	85	55	U
126	86	56	V
127	87	57	W
130	88	58	X
131	89	59	Y
132	90	5A	Z
133	91	5B	[
134	92	5C	\
135	93	5D	]
136	94	5E	^
137	95	5F	_

<i>oct</i>	<i>dec</i>	<i>hex</i>	<i>char</i>
140	96	60	`
141	97	61	a
142	98	62	b
143	99	63	c
144	100	64	d
145	101	65	e
146	102	66	f
147	103	67	g
150	104	68	h
151	105	69	I
152	106	6A	j
153	107	6B	k
154	108	6C	l
155	109	6D	m
156	110	6E	n
157	111	6F	o

<i>oct</i>	<i>dec</i>	<i>hex</i>	<i>char</i>
160	112	70	p
161	113	71	q
162	114	72	r
163	115	73	s
164	116	74	t
165	117	75	u
166	118	76	v
167	119	77	w
170	120	78	x
171	121	79	y
172	122	7A	z
173	123	7B	{
174	124	7C	
175	125	7D	}
176	126	7E	~
177	127	7F	del

Index

Symbols

- - decrement operator, 142–43
- - unary minus operator, 132–33
- '
 - single quote, 64
 - single quote
 - printing, 68
- !
 - logical negation operator, 146
- !=
 - not equal to operator, 96, 145
- "
 - double quotes, 320
 - surrounding filenames, 320
- #
 - preprocessor symbol, 53
 - trigraph sequence, 69
- ## operator, 315
- %
 - remainder operator, 134–42
- %c format specifier, 50
- %d format specifier, 50, 272
- %f format specifier, 50
- %o format specifier, 50
- %p print specifier, 86
- %s format specifier, 50, 186
- %x format specifier, 50
- &
 - address of operator, 85, 161, 272
 - in scanf() calls, 52
 - bitwise AND operator, 150
- &&
 - logical AND operator, 146
- &=
 - AND assign operator, 157
- ()
 - cast operator, 158
 - function call, 47
 - macro calls, 307
 - parenthesized expressions, 128–30
- *
 - dereference operator, 87, 161
- */, end comment, 45
- ,
 - comma operator, 143–44
- .
 - decimal point, 69
 - dot operator, 161
- ...

- ellipsis, 283
 - .c filename extension, 34
 - .h filename extension, 35
 - /*, begin comment, 45
 - :
 - bit fields, 247–51
 - statement label, 121
 - trigraph sequence, 69
 - ;
 - mistakenly used to end macro definition, 307
 - ; semicolon
 - misplaced, 95
 - ?
 - conditional operator, 160
 - []
 - array operator, 161
 -]
 - trigraph sequence, 69
 - ^
 - bitwise exclusive OR operator, 150
 - trigraph sequence, 69
 - ^=
 - XOR assign operator, 157
 - __DATE__ macro, 312
 - __FILE__ macro, 312
 - __LINE__ macro, 312
 - __STDC__ macro, 312
 - __TIME__ macro, 312
 - _IOFBF constant, 343
 - _IOFBF macro, 419
 - _IOLBF constant, 343
 - _IOLBF macro, 419
 - _IONBF constant, 343
 - _IONBF macro, 419
 - {
 - trigraph sequence, 69
 - }
 - compound statement, 98–99
 - missing braces, 102
 - function body, 36
 - initializing arrays, 168
 - initializing structures, 239
 - missing in compound statement, 102
 - |
 - bitwise inclusive OR operator, 150
 - trigraph sequence, 69
 - ||
 - logical OR operator, 146
 - |=
 - OR assign operator, 157
 - }
 - trigraph sequence, 69
 - ~
 - bitwise complement operator, 150
 - trigraph sequence, 69
 - +
 - unary plus operator, 132–33
 - ++
 - increment operator, 142–43
 - <
 - less than operator, 96, 145
 - <<
 - left shift operator, 150
 - <<=
 - left shift assign operator, 157
 - <=
 - less than or equal to operator, 96, 145
 - =
 - equal to operator, 96, 145
 - >
 - right arrow operator, 161
 - >
 - greater than operator, 96, 145
 - >=
 - greater than or equal to operator, 96, 145
 - >>
 - right shift operator, 150
 - >>=
 - right shift assign operator, 157
- A—**
- \a
 - alert, 68
 - abort signal, 394

- abort()* function, 380, 394, 428
- aborting a program, 123
- abs()* function, 431
- absolute values
 - abs()* function, 431
 - fabs()* function, 391
 - labs()* function, 431
- abstraction, 22
 - of programming, 18
- access modes, 332, 400
- acos()* function, 389
- actual arguments*, 36, 270, 271
 - compatibility, 279
 - conversion of, 276
- Ada programming language, 368
- additive operators, 129, 133
- address of operator
 - &, 85
- address of operator &*, 161, 272
 - in *scanf()* calls, 52
- addresses
 - finding, 85–86
 - of variables, 85
 - passing as arguments, 272
 - storing, 86
 - writing, 86
- aggregate data types, 57
- Aho, 368
- alert
 - escape sequence `\a`, 68
- algorithm*, 32
- algorithms, 14, 179–81
 - choosing efficient, 365
 - sorting, 179, 291
- alignment
 - natural, 246
 - structure member, 245–47
- allocation. *See* memory allocation
- allusions, 222–24
- alpha test, 27
- alphabets
 - for different languages, 383
- American National Standards Institute.
 - See* ANSI Standard
- American Standard Code for Information Interchange. *See* ASCII character set
- ampersand. *See* address-of operator
- analysis
 - lexical, 452
 - performance, 28
- AND operator
 - bitwise &, 150
 - logical &&, 146
- AND-assign operator &=, 157
- ANSI standard, 6–7
 - differences from K&R standard, 452–61
 - rationale of, 6
- arc cosine
 - acos()* function, 389
- arc sine
 - asin()* function, 389
- arc tangent
 - atan()* function, 389
- architectural design, 13, 361–62
- argc*, 296
- argument type, 37
- arguments, 36
 - actual, 36, 270
 - data type of, 37
 - declaring, 275–78
 - formal, 36, 270
 - multidimensional arrays, 199–200
 - passing, 270–72
 - passing arrays, 177–79, 253
 - passing pointers, 173–74
 - passing structures, 252–53
 - to macros, 308
 - variable number of, 395–97
- argv*, 296
- arithmetic assignment operators, 136–39
- arithmetic data types, 57
- arithmetic functions
 - integer, 431–32
- arithmetic shift, 152
- array elements
 - accessing with pointers, 174–76
 - referencing, 165
- array name

- interpretation of, 176
 - naked, 176
 - array names, 164, 165
 - array operator [], 161
 - array size, 165
 - arrays, 164–211
 - base address of, 176
 - bounds checking, 181
 - declaring, 165–66
 - finding number of elements in. *See*
 - initial element, 165
 - initializing, 168
 - memset()* function, 433
 - multidimensional, 194–201
 - initializing, 197–98
 - passing as function arguments, 199–200
 - referencing elements, 199
 - storage, 197
 - nonoverlapping, 283
 - of **chars**. *See* strings
 - of pointers, 202–5
 - of structures, 241–42
 - passing as function arguments, 177–79
 - size of, 165
 - storage of, 167–68
 - syntax of declaration, 165
 - ASCII character set, 63
 - ASCII codes, 473
 - asctime()* function, 442
 - asin()* function, 389
 - assembler*, 2
 - assembly language*, 2
 - assert()* function, 380
 - assert.h* header file, 158, 380
 - assignment, 38
 - string, 183–85
 - assignment conversions. *See*
 - assignment operator =
 - confusing with equal to operator, 96
 - erroneous use in macro definitions, 309
 - assignment operators
 - arithmetic, 136–39
 - bitwise, 157
 - assignment statements, 43
 - assignment suppression flag
 - in *scanf()* function, 415
 - assignments
 - multiple, 137
 - associativity
 - of operators, 128–32
 - associativity of operators, 129
 - atan()* function, 389
 - atexit()* function, 428
 - atof()* function, 423
 - atoi()* function, 424
 - atol()* function, 424
 - auto** keyword, 213, 230
 - automatic build, 20
 - automatic conversions, 72–79
 - automatic duration, 213
- B—
- \b**
 - backspace, 68
 - B programming language, 5
 - back-end, 14
 - backslash, 69
 - backslash
 - printing, 68
 - backspace
 - escape character sequence **\b**, 68
 - bar
 - trigraph sequence, 69
 - base address
 - of arrays, 176
 - base type
 - of bit fields, 248
 - BCD form, 269
 - beta test, 27
 - binary format, 328
 - binary numbers*, 1
 - binary operators, 73, 97
 - binary stream modes, 332

- binary tree, 365
 - binder. *See* linker
 - binding. *See* associativity of operators
 - bit, 1
 - bit fields, 247–51
 - base type of, 248
 - declaring, 248
 - memory allocation of, 248
 - portability of, 250
 - bit-manipulation operators, 150–57
 - bitwise AND operator &, 150
 - bitwise assignment operators, 157
 - bitwise complement operator ~, 150
 - bitwise exclusive OR operator ^, 150
 - bitwise inclusive OR operator |, 150
 - block buffering, 330
 - block scope, 217, 219–20
 - blocks, 329
 - Boole, George, 97
 - Boolean data types, 97
 - Boolean expressions, 97
 - bounds checking, 181
 - braces. *See* compound statements
 - left
 - trigraph sequence, 69
 - right
 - trigraph sequence, 69
 - brackets
 - left, 69
 - right
 - trigraph sequence, 69
 - break statement, 120–21
 - to exit switch, 104
 - bridge metaphor, 29–30
 - Brodie, James, 6
 - Brooks, Fredrick P., 17
 - Brooks's law, 18
 - bsearch()* function, 430
 - bubble sort, 179
 - buffer, 329
 - keyboard, 109
 - buffering, 329–30
 - block, 330
 - buffer manager, 330
 - line, 330
 - buffers
 - flushing, 330
 - setting size of
 - setbuf()* function, 343
 - bug alerts
 - binding of macro arguments, 314
 - comparing floating-point values. *See* confusing = with ==, 96
 - confusing **typedef** with **#define**, 83
 - dual meanings of **static**. *See* ending a macro definition with a semicolon, 307
 - integer division. *See* misplaced semicolons, 95
 - missing braces, 102
 - mixing signed and unsigned types, 80
 - no nested comments, 48
 - off-by-one errors. *See* opening a file, 335
 - passing structures. *See* referencing elements in a multidimensional array, 199
 - side effects, 150
 - side effects in macro arguments, 311
 - space between left parenthesis and macro name, 310
 - the dangling **else**, 101
 - unexpected expression evaluation, 132
 - using = to define a macro, 309. *See* walking off the end of an array, 181
 - bugs, 25
 - build file, 20
 - built-in macros, 312
 - Built-In Macros, 313
- C—
- C interpreter
 - software engineering case study, 355–75
 - C programming language
 - history of, 5
 - nature of, 8
 - C++, 7, 368

- calculator program, 118–20
- call by reference, 270
- call by value, 270
- calloc()* function, 232, 427
- caret
 - trigraph sequence, 69
- carriage return
 - escape character sequence `\r`, 68
- cascading if statement. *See* nested if statements
- CASE. *See* Computer-Aided Engineering
- case label, 102
- case labels
 - maximum number of, 448
- case sensitivity. *See*
- case-mapping functions, 381
 - runtime library, 382
- cast operator `()`, 158
 - confusing with unions. *See*
- casts, 80–81
 - of function return values, 233
 - to `void`. *See*
 - to `volatile`, 229
- `cc` command, 34
- ceil()* function, 390
- `char` type specifier, 63
- `CHAR_BIT` macro, 449
- `CHAR_MAX` macro, 449
- `CHAR_MIN` macro, 449
- character arrays. *See* strings
- character constants, 64
- character handling
 - character testing functions, 381
 - runtime library, 381–82
- character set
 - EBCDIC. *See* EBCDIC character set
- character sets
 - ASCII. *See* ASCII character set
- character-handling functions
 - case-mapping functions, 382
- characteristic
 - of floating-point constants, 71
- characters, 63–65
 - alphabetic, 382
 - control, 382
 - lowercase, 382
 - multibyte, 65
 - printable, 382
 - uppercase, 382
- character-testing functions, 381
- chars
 - vs. strings, 185–86
- checksum* function, 124
- Clean C, 7
 - reserved keywords, 42
- clearerr()* function, 331, 397
- clock()* function, 442
- clock_t* type, 441
- `CLOCKS_PER_SEC` macro, 441
- closing files, 335
- coding stage, 24–25
- collating sequences, 195
- comma operator `,`, 143–44
- command line arguments, 296
- command prompt, 34
- comments, 45–46
 - no nesting, 48
- comparing floating-point values, 144
- comparison expressions, 96–98
- comparison operators, 96
- compilation
 - conditional, 316
 - syntax of, 316
 - in UNIX environment, 34
- compiler, 3
- compile-time errors, 78
- compiling source files, 33
- complement operator
 - bitwise `~`, 150
- compound statements, 98–99
- Computer-Aided Software Engineering (CASE), 369
- conditional branching, 92–107
- conditional compilation, 316

- syntax of, 316
- conditional operator `?:`, 160
- const** pointers, 227
- const** storage-class modifier, 231. 226–27
- constant
 - string, 182
- constant expressions, 127
- constants, 39–40
 - character, 64. *See*
 - decimal, 65
 - enumeration, 81
 - floating-point, 71
 - hexadecimal, 65
 - integer, 65–69
 - naming, 54
 - octal, 65
 - unsigned, 67
- continuation character, 51
- continue statement, 120–21
- control characters, 158, 382
- control flow, 92–125
- conversion character
 - in *printf()* function, 407
- conversion characters
 - in *printf()* function, 408
 - in *scanf()* function, 416
- conversion specifiers
 - in *printf()* function, 407
- conversions
 - automatic, 72–79
 - casts, 80–81
 - floating-point to floating-point, 77–79
 - floating-point to integer, 77–79
 - implicit, 73–75
 - in expressions, 73–75
 - integer to floating-point, 77–79
 - signed and unsigned, 80
 - unsigned, 76–77
- conversions.
 - integral widening, 73
- cos()* function, 389
- cosh()* function, 389
- ctime()* function, 443
- CTRL-C control character, 123

- ctype.h* header file, 378
- currency symbol, 385

—D—

- Dam, A Van, 158
- dangling **else**, 96, 101
- data
 - reading, 335–40
 - writing, 335–40
- data structures, 14
 - choosing efficient, 365
- data type, 57
 - integer, 60–65
- data type qualifiers, 58
 - signed, 63
 - unsigned, 63
- data types
 - aggregate, 57
 - arithmetic, 57
 - character, 63–65
 - enumeration, 81–83
 - floating-point, 69–71
 - hierarchy of, 58
 - int, 60
 - long int**, 61
 - mixing, 72–79
 - of functions, 59
 - scalar, 57–91
 - hierarchy of, 74
 - keywords, 59
 - short int**, 61
 - void**, 57, 83–84
- date and time functions, 441–46
- Date, C. J., 15
- dates
 - different ways of displaying, 383
- DBL_DIG* macro, 451
- DBL_EPSILON* macro, 450
- DBL_MAX* macro, 450
- DBL_MIN* macro, 450
- debuggers, 26
- debugging, 25, 372
- debugging code

- adding to source files, 219
- decimal constants, 65
- decimal point, 69
 - different representations of, 385
 - representing in different ways, 383
- decimal point character, 383
- declarations, 58–65
 - allusions, 222–24
 - array, 165–66
 - complex, 297–303
 - composing, 299
 - deciphering, 298
 - definitions, 222–24
 - forward referencing, 245
 - string, 182–83
- decrement operators, 142–43
 - precedence of, 142
- decryption, 169–70
- default label, 102
- #define** directive, 305–15, 54, 305–15
 - confusing with Typedef, 83
- defined** preprocessor operator, 319
- definitions, 222–24
 - tentative, 223
- dependency lines, 371
- dereference operator
 - ***, 87
- dereference operator *****, 161
- design
 - detailed, 365–69
- detailed design, 365–69
- device registers, 228
- diagnostics
 - runtime library, 380–81
- difftime()* function, 443
- Dijkstra, E. W., 122
- div()* function, 431
- div_t* type, 423
- division
 - integer, 137
- do...while** statement, 109–10
 - syntax, 110
- documentation, 372–73

- domain errors, 387
- dot operator **.**, 161
- double quotes, 182
 - printing, 68
 - surrounding filenames, 320
- double** type specifier, 69
- double-precision, 69
- doubly linked lists, 256
- duration, 213–17
 - automatic, 213
 - definition, 212
 - fixed, 213, 216–17
- dynamic memory allocation, 231–35

—E—

- EBCDIC character set, 63
- echo* command, 296
- ecvt()* function, 387
- EDOM** macro, 387
- efficiency, 4, 365
 - and arrays, 165
 - and bit fields, 250
 - and buffering, 329
 - and readability, 190, 204
 - and recursion, 296
 - and **volatile** specifier, 229
 - choosing algorithms, 365
 - choosing data structures, 365
 - of sorting algorithms, 179
 - returning pointers to structures, 254
 - strength reduction, 209
 - using arithmetic assignment operators, 305–15
 - using pointers to pointers, 208
- element. *See* array elements
- #elif** directive, 316
- #else** directive, 316
- ellipsis
 - ..., 283
- else**
 - dangling, 96, 101
- else if** statement, 99
- else** keyword, 93

empty statements. *See* null statements

encryption, 169–70

#endif directive, 316

end-of-file flag, 331

engine, 14

entropy, 28

enum type specifier, 81–83, 267

enumeration type, 81–83

enumeration variables, 267–69

environment functions, 428–30

EOF, 108, 331

epsilon, 123, 450

equal to operator **==**, 96, 145

ERANGE macro, 387

errno, 351, 378

errno macro, 380

errno variable, 332

errno.h header file, 378

errno.h header file., 332

#error directive, 323

error flag, 331

error handling

- I/O, 331
- runtime library, 380

error messages, 380, 406, 435, 461

escape character sequences, 67–68

escape sequence, 51

evaluation

- order of, 130, 131–32

executable code, 33

executable program, 34

- loading, 35

executable statement, 38

exit() function, 47, 429

EXIT_FAILURE macro, 423

EXIT_SUCCESS macro, 423

exp() function, 388

exponent, 71

exponential functions

- runtime library, 388

exporting information, 17

expressions, 43–44, 126–63

- Boolean, 97
- comparison, 96–98
- constant, 127
- conversions in, 73–75
- float, 127
- integral, 127
- omitting in **for** statement, 114–15
- pointer, 127
- relational

 - side effects, 150

Extended Binary-Coded Decimal Interchange Code. *See* EBCDIC character set

extern keyword,

extern specifier, 222, 230, 281

—F—

⌘

- form-feed, 68

fabs() function, 391

fclose() function, 335, 397

fcvt() function, 387

feof() function, 331, 398

ferror() function, 331, 398

fflush() function, 398

fgetc() function, 336, 399

fgetpos() function, 399

fgets() function, 338, 399

field widths

- in *printf()* function, 411
- in *scanf()* function, 415

fields. *See* structure members

file management functions, 352

file pointer, 327, 333

file position indicator, 328, 333, 337, 344, 353, 399, 400, 402, 403, 404, 418, 421

file scope, 217, 220–21

FILE structure, 327

filenames

- .c extension, 34
- .h extension, 35
- files
 - closing, 332–35
 - opening, 332–35
- FILO, 269
- first in, last out (FILO) queues, 269
- fixed duration, 212, 213, 216–17
 - initialization of variables with, 216
- flag characters
 - in *printf()* function, 410
- float constants, 71
- float expressions, 127
- float type specifier, 69
- float.h* header file, 378
- floating-point data types, 69–71
 - characteristics of, 450
 - mixing with integers, 77–79
- floating-point exception signal
 - SIGFPE* macro, 394
- floating-point overflow, 387
- floating-point values
 - comparing, 144
- floor()* function, 391
- FLT_DIG* macro, 451
- FLT_EPSILON* macro, 450
- FLT_MAX* macro, 450
- FLT_MIN* macro, 450
- flushing buffers, 330, 397
- fmod()* function, 391
- Foley, 15
- fopen()* function, 328, 332, 400
- for statement, 110–14
 - null statements, 115–16
 - omitting expressions, 114–15
 - syntax, 111
- formal arguments, 36, 270
- format modifiers
 - in *printf()* function, 407
- format specifiers, 50
- format string, 50
- formatting source files, 44–46
- form-feed
 - escape character sequence `\f`, 68
- forward referencing, 245
- fprintf()* function, 401
- fputc()* function, 336, 402
- fputs()* function, 338, 402
- fread()* function, 339, 402
- free()* function, 232, 427
- freopen()* function, 403
- frexp()* function, 388
- front-end, 14
- fscanf()* function, 403
- fseek()* function, 344, 403
- fsetpos()* function, 404
- tell()* function, 344, 404
- function allusions, 272, 278–82
 - syntax, 281
- function body, 36, 276
- function calls, 272, 282–83
 - syntax, 284
- function definitions, 272
 - prototype form, 273
 - syntax, 273–74
 - traditional form, 274
- function invocation. *See* function calls
- function names, 36
 - choosing, 275
 - reserved, 377
- function prototypes, 37
- function scope, 217, 220
- functions, 21, 35–38
 - anatomy of, 36–38
 - calling, 272–84
 - calling with pointers, 286
 - declaring, 272–84
 - pointers to, 284–93
 - returning, 291
 - return type, 275
 - return type agreement, 286
 - return type of
 - declaring, 59
 - vs. macros, 311, 379
- fwrite()* function, 339, 405

—G—

- garbage values, 167, 215
- gcvt()* function, 387
- general utilities
 - runtime library, 422–32
- generic pointers, 233
- getc()* function, 336, 405
- getchar()* function, 405
- getenv()* function, 429
- gets()* function, 192, 406
- global variables, 217, 220–24
 - declaring, 224
- gmtime()* function, 443
- goto* labels
 - scope of, 220
- goto* statement, 121–22
- greater than operator >, 96, 145
- greater than or equal to operator >=, 96, 145
- grouping
 - of operators, 128

—H—

- hardware, 1
- hash table, 366
- header files, 35, 371
 - runtime library, 377–378
- heap sort algorithms, 291
- hexadecimal constants, 65
- high-level programming languages, 3
- hobgoblin
 - of software engineers, 22
- holes
 - in structures, 245
- Hopcroft, 368
- Hopper, Lieutenant Grace, 25
- horizontal tab
 - escape character sequence `\t`, 68
- HUGE_VAL* macro, 387

- hyperbolic cosine
 - cosh()* function, 389
- hyperbolic functions
 - runtime library, 387
- hyperbolic tangent
 - tanh()* function, 389

—I—

- I/O, 326–54
 - binary format, 328
 - error handling, 331
 - error-handling functions, 351
 - file management functions, 352
 - once character at a time, 336
 - one block at a time, 339
 - one line at a time, 338
 - random access, 344–53
 - runtime library functions, 397–422
 - selecting a method, 341–43
 - text format, 328
 - to files, 352
 - unbuffered, 343
- I/O functions
 - runtime library, 397–422
- idempotent header files, 377
- identifier. *See* names
- ldiv()* function, 432
- if* statement, 92–95
 - nested, 99
 - syntax, 93
- #if* directive, 316
- #ifdef* directive, 319
- #ifndef* directive, 319
- illegal instruction signal
 - SIGILL* macro, 394
- implementation limits, 447–51
- implicit conversions, 73–75
- include facility, 320
- #include* directive, 53, 320
- increment operators, 142–43
 - precedence of, 142
- indentation, 52
 - misleading, 102

- with nested loops, 101, 117
- index, 164
- infinite loops, 122–25
- information hiding, 17, 368–69
- initial element, 165
- initialization, 71–72
 - global variable, 321
 - multidimensional array, 197–98
 - pointer, 89
 - structure, 239–40
 - union, 265–66
- initializing arrays, 168
- input and output, 326–54
- instruction set*, 1
- int type specifier, 36, 58, 60
- INT_MAX* macro, 449
- INT_MIN* macro, 449
- integer arithmetic functions, 431–32
- integer constants, 65–69
- integer overflow, 76
- integer types
 - mixing with floating-point types, 77–79
- integers, 60–65
 - dividing, 137
 - mixing, 75–76
 - signed, 63
 - unsigned, 62
- integral expressions, 127
- integral promotion. *See* integral widening conversions
- integral widening conversions., 73
- interface, 17
- interpreter
 - software engineering case study, 355–75
- interrupt signal
 - SIGINT* macro, 394
- intialization, 71–72
- IO
 - to *stdin* and *stdout*, 350
- isalnum()* function, 382
- isalpha()* function, 382

- isctrl()* function, 382
- isdigit()* function, 382
- isgraph()* function, 382
- islower()* function, 382
- ISO standard, 6–7
- isprint()* function, 382
- ispunct()* function, 382
- isspace()* function, 115, 382
- isupper()* function, 382
- isxdigit()* function, 382
- iteration. *See* looping

—J—

- jmp_buf* variable, 391
- jumps
 - nonlocal, 391–93
- justification
 - in *printf()* function, 407

—K—

- K&R standard, 5
 - differences from ANSI standard, 452–61
- Kanji chracter set, 63
- Kernighan, Brian, 5, 19, 45
- keywords, 36, 42
 - reserved, 41
- Knuth, Donald E., 368

—L—

- labels
 - case*, 102
 - goto*, 217
 - statement, 121
- labs()* function, 431
- LC_ALL* macro, 383
- LC_COLLATE* macro, 383
- LC_CTYPE* macro, 383

- LC_MONETARY* macro, 383
 - LC_NUMERIC* macro, 383
 - LC_TIME* macro, 383
 - lconv* structure, 383
 - LDBL_DIG* macro, 451
 - LDBL_EPSILON* macro, 450
 - LDBL_MAX* macro, 450
 - LDBL_MIN* macro, 450
 - ldexp()* function, 388
 - ldiv()* function, 432
 - ldiv_t* structure, 423
 - leaves, 130
 - left brace
 - trigraph sequence, 69
 - left bracket, 69
 - left-shift-assign operator `<<=`, 157
 - left shift operator `<<`, 150
 - less than operator `<`, 96, 145
 - less than or equal to operator `<=`, 96, 145
 - lexical analysis, 40, 452
 - library. *See* runtime library
 - limits
 - implementation, 447–51
 - numerical, 448
 - translation, 447
 - limits.h* header file, 70, 378
 - `#line` directive, 321
 - line buffering, 330
 - line control, 321
 - linked lists, 256–62
 - adding elements, 258–59
 - creating elements, 257–58
 - deleting elements, 260–61
 - doubly, 256
 - finding elements, 261
 - inserting elements, 259–60
 - linker, 33, 34
 - linking
 - in UNIX environment, 34
 - linking object files, 34
 - lint* utility, 369
 - LISP programming language, 295
 - loader, 33, 35
 - local variables, 213
 - locale parameters
 - runtime library, 383–84
 - locale.h* header file, 378, 383
 - localeconv()* function, 384
 - localtime()* function, 443
 - log()* function, 389
 - log10()* function, 389
 - logarithmic functions
 - runtime library, 388
 - logical AND operator `&&`, 146
 - logical bitwise operators, 152–57
 - logical negation operator `!`, 146
 - logical operators, 146–49
 - truth table for, 147
 - logical OR operator `||`, 146
 - logical shift, 152
 - long double** constants, 71
 - long double** type specifier, 69
 - long int** data type, 61
 - LONG_MAX* macro, 449
 - LONG_MIN* macro, 449
 - longjmp()* function, 392
 - looping, 107–16
 - loops
 - infinite, 122–25
 - nested, 116–18
 - lvalue*, 43
- M—
- machine language, 1, 2
 - macro arguments, 308
 - macro body, 305
 - macro definition
 - removing, 310
 - macro expansion, 305
 - macro names, 305
 - macros, 304–15
 - syntax of, 306

- advantages of, 311
 - built-in, 312
 - disadvantages of, 312
 - recursive, 310
 - string producer, 313
 - testing existence of, 319
 - token pasting, 315
 - vs. functions, 311, 379
 - mailbox analogy for computer memory, 39
 - main()* function, 46–50, 46–50
 - maintainability, 4
 - and formatting, 201
 - and intermodule dependencies, 15
 - and performance, 221
 - and relational expressions, 149
 - avoiding global variables, 221
 - avoiding intermodule dependencies, 362
 - changing parameter values, 242
 - division into subsystems, 15
 - limiting scope, 219
 - naming conventions, 41
 - using macros, 306
 - maintenance, 28–29
 - make* utility, 369, 371
 - makefiles, 371
 - malloc()* function, 232, 427
 - mantissa, 71
 - masking, 154
 - math.h* header file, 378, 387
 - mathematics functions
 - runtime library, 385–91
 - MB_CUR_MAX* macro, 423
 - mblen()* function, 439
 - mbstowcs()* function, 440
 - mbtowc()* function, 440
 - members. *See* structure members
 - memchr()* function, 432
 - memcmp()* function, 432
 - memcpy()* function, 433
 - memmove()* function, 433
 - memory
 - mailbox analogy, 39
 - memory allocation
 - dynamic, 231–35
 - for strings, 182
 - of array elements, 167
 - of array parameters, 177
 - of automatic variables, 213
 - of enums, 83
 - of integers, 60
 - memory management functions
 - runtime library, 426–28
 - memory operators, 161–63
 - memset()* function, 433
 - merge sort algorithm, 291
 - milestones, 363, 364
 - minus operator, 73
 - mktime()* function, 444
 - modf()* function, 390
 - modules, 15, 361
 - modulus operator. *See* remainder operator
 - most significant bit, 61
 - multibyte character functions, 439–41
 - multibyte characters, 65
 - multidimensional arrays, 194–201
 - initializing, 197–98
 - passing as function arguments, 199–200
 - referencing elements, 199
 - multiple assignments, 137
 - multiplicative operators, 133
- N—
- \n*
 - newline, 68
 - naked array name, 176
 - name spaces
 - structure, 244
 - union, 244
 - names, 40–42
 - array, 167
 - choosing, 42
 - legal and illegal, 41
 - length of, 41

- reserved, 463
- naming conventions, 41
- natural alignment, 246
- NDEBUG* macro, 380
- negative numbers
 - representation of, 61
- nested comments, 48
- nested if statements, 99
- nested loops, 116–18
- nested structures, 244
- newline
 - escape character sequence `\n`, 68
- newline character, 49
- newlines, 44
 - ending preprocessor directives, 53
 - escape sequence `\n`, 51
- NIH syndrome, 19
- node, 130
- nonlocal jumps
 - runtime library, 391–93
- not equal to operator `!=`, 96, 145
- not invented here (NIH) syndrome, 19
- NOT operator
 - logical `!`, 146
- NP-complete problems, 375
- NULL*, 331
- null character, 182
- NULL* macro, 378, 380
- null pointer, 172–73, 172, 331
- null statements, 115–16
- numerical limits, 448

—O—

- object code, 33
- object files
 - linking, 34
- objects, 368
- octal constants, 65
- off-by-one errors, 114

- offsetof* macro, 247, 269
- omitted-*extern* strategy, 224
- operands, 73
- operating systems, 5
- operators, 126–63
 - additive, 133
 - address of `&`, 52, 161
 - assignment, 136–39
 - associativity of, 128–32
 - binary, 73, 97
 - bit manipulation, 150–57
 - bitwise
 - logical, 152–57
 - comparison, 96
 - decrement, 142–43
 - precedence of, 142
 - increment, 142–43
 - precedence of, 142
 - logical, 146–49
 - logical bitwise, 152–57
 - memory, 161–63
 - multiplicative, 133
 - postfix, 139
 - precedence of, 128–32
 - prefix, 140
 - shift, 151–52
 - sizeof*, 158–59
 - unary, 73, 132–33
- optimizations. *See also* efficiency
 - performed by compiler, 228, 229
 - strength reduction, 241
- OR-assign operator `|=`, 157
- OR operator
 - bitwise exclusive `^`, 150
 - bitwise inclusive `|`, 150
 - logical `||`, 146
- order of evaluation, 130, 131–32
- output. *See* I/O
- overflow
 - floating-point, 387
 - integer, 76
- overflows, 78

—P—

- padding

- in *printf()* function, 51
- parameters. *See* arguments
- parentheses, 128–30
- parsing, 40
- pass by reference, 252
- pass by value, 252
- passing arguments, 270–72
- pattern matching, 190–93
- PCC (Portable C Compiler), 6
- PDL. *See* pseudocode
- performance analysis, 28
- perror()* function, 406
- planning software projects, 17
- Plauger, P. J., 19, 45
- plus operator +, 132
- pointer arithmetic, 171–73
- pointer expressions, 127
- pointer subtraction, 171–72
- pointers, 164–211
 - arithmetic, 171–73
 - arrays of, 202–5
 - dereferencing, 87–89
 - generic, 233
 - initializing, 89
 - introduction to, 86–90
 - null, 172–73, 172
 - passing as function argument, 173–74
 - to functions, 284–93
 - assigning values to, 285
 - invoking, 286
 - to pointers, 205–11
 - to strings, 204
 - using to access array elements, 174–76
- popping
 - off a stack, 269
- portability, 3
 - accessing the file position indicator, 328
 - and bit fields, 248, 250
 - and I/O, 326
 - and integer division, 137
 - and structure allocation, 246
 - and variable argument lists, 395
 - declaring global variables, 225
 - dereferencing pointers to functions, 287
 - using *sizeof*, 180, 348
 - variable addresses, 86
- postfix operators, 139
- pound sign #, 69
- pow()* function, 390
- #pragma** directive, 323
- precedence
 - of decrement operators, 142
 - of increment operators, 142
 - of operators, 128–32
- precision, 69
 - loss of, 77
- precision specifier
 - in *printf()* function, 412
- prefix operators, 140
- preprocessor, 52–54, 304
- printf()* function, 50, 407–13
- private types, 368
- procedure, 35
- product specification, 12–13, 357–61
- profiling, 28
- program design language, 24
- program development, 31–35
- program execution
 - terminating, 47
- program maintenance, 28–29
- program scope, 217, 220–21
- programming languages
 - high-level, 2, 3
 - systems, 5
- programming style, 356
- programming tools, 19
- project planning, 17–21, 362–64
- Prolog programming language, 295
- prototypes, 37, 272, 278–82
 - syntax, 281
 - variable argument lists, 283
- pseudocode, 24
- pushing
 - onto a stack, 269

putc() function, 336, 413

putchar() function, 413

puts() function, 414

—Q—

qsort() function, 430

qualifiers. *See* data type qualifiers

quicksort algorithm, 291

quiet conversions. *See* automatic conversions

quotes

double, 320

printing, 68

single

printing, 68

quotient

div() function, 431

—R—

\r

carriage return, 68

raise() function, 395

rand() function, 426

RAND_MAX macro, 423

random access I/O, 344–53

random numbers

pseudo

runtime library, 426

range errors, 387

readability, viii, 3, 28

and efficiency, 190

and formatting, 44, 201

choosing function names, 275

declaring functions, 60

formatting comments, 45

limiting scope, 219

nested structures, 243

using arithmetic assignment operators, 138

using parentheses, 129

using **switch** statements, 101

reading data, 335–40

reading strings, 186–87

realloc() function, 232, 427

record, 236

variant, 236

recursion, 293

of macros, 460

redundancy

eliminating, 22

needless, 22

register storage class, 225–26, 230, 276

relational expressions

side effects, 150

relational operators. *See* comparison operators

remainder

of integer division, 137

remainder operator **%**, 134–42

remove() function, 414

rename() function, 414

reserved keywords, 36, 41, 42

Clean C, 42

reserved names, 463

return statement, 254, 277, 278

return type

of function, 275

return values, 277

in recursive calls, 294

type agreement, 286

rewind() function, 414

right-arrow operator **->**, 161

right brace

trigraph sequence, 69

right bracket

trigraph sequence, 69

right-shift-assign operator **>>=**, 157

right shift operator **>>**, 150

Ritchie, Dennis M., 5

ROMs, 227

rounding

of floating-point numbers, 73, 413

runtime errors, 78

running out of stack memory, 294

runtime library, 35, 376–446
 character handling, 381–82
 date and time functions, 441–46
 diagnostics, 380–81
 environment functions, 428–30
 error handling, 380
 function names, 377
 general utilities, 422–32
 header files, 377
 header filese
 list of, 378
 I/O functions, 397–422
 integer arithmetic functions, 431–32
 mathematics functions, 385–91
 memory management functions, 426–
 28
 multibyte functions, 439–41
 nonlocal jumps, 391–93
 random number generation, 426
 searching and sorting functions, 430–
 31
 setting locale parameters, 383–84
 signal handling, 393–95
 string conversion functions, 423–26
 string handling functions, 432–39
 synopses, 377–79
 variable argument lists, 395–97
 rvalue, 43

—S—

scalar data types, 57–91
 hierarchy of, 74
 keywords, 59
 scaling
 in pointer arithmetic, 171
 turning off with a cast, 196
scanf() function, 49, 52, 415–18
SCHAR_MAX macro, 449
SCHAR_MIN macro, 449
 scientific notation, 70–71
 scope, 217–21
 block, 217, 219–20
 definition, 212
 file, 217, 220–21
 function, 217, 220
 hierarchy, 218

 program, 217, 220–21
 searching and sorting funtions, 430–31
SEEK_CUR macro, 344
SEEK_END macro, 344
SEEK_SET macro, 344
 segment violation signal
 SIGSEGV macro, 394
 self-referencing structures, 244–45
 semicolon ;
 misplaced, 95
 mistakenly used to end macro
 definition, 307
setbuf() function, 343, 419
setjmp() function, 392
setjmp.h header file, 378, 391
setlocale() function, 384
setvbuf() function, 343, 419
 shift
 arithmetic, 152
 logical, 152
 shift operators, 151–52
 Shore, John, 10
 short-circuit evaluation, 150
 short int data type, 61
 short type specifier, 58
SHRT_MAX macro, 449
SHRT_MIN macro, 449
 side effects
 in expressions, 141
 in macro arguements, 311
 in relational expressions, 150
SIG_DFL macro, 394
SIG_ERR macro, 394
SIG_IGN macro, 394
SIGABRT macro, 394
SIGFPE macro, 394
SIGILL macro, 394
SIGINT macro, 394
 sign bit, 61
 sign extension, 75
 signal handling

- runtime library, 393–95
- signal()* function, 393
- signal.h* header file, 378, 393
- signed integers, 63
- signed type specifier, 58, 63
- sign-preserving conversions, 77
- SIGSEGV* macro, 394
- SIGTERM* macro, 394
- sin()* function, 389
- single quote
 - printing, 68
- single quotes, 64
- sinh()* function, 389
- size_t* macro, 247
- size_t* type, 339, 423
- sizeof* operator, 158–59, 167, 348
 - using for portability, 180
- smallest integer
 - ceil()* function, 390
- software, 1
- software engineering
 - architectural design, 361–62
 - case study, 355–75
 - debugging, 372
 - detailed design, 365–69
 - documentation, 372–73
 - information hiding, 368
 - introduction to, 9–30
 - product specification, 357–61
 - software tools, 369–72
 - source management, 370
- software engineering
 - project planning, 362–64
- software production, 10
- software tools, 369–72
- sort routine
 - generalized, 287
- sorting algorithms, 179, 291
 - bubble sort, 179
- sorting functions
 - runtime library, 430–31
- source files
 - compiling, 33
 - formatting, 44–46
- source management, 19, 370
- sprintf()* function, 420
- sqrt()* function, 390
- srand()* function, 426
- sscanf()* function, 420
- stack, 269
- stack memory
 - running out of, 294
- standard streams, 328
- statement label, 121
- statements
 - compound, 98–99
- static** storage class specifier, 168, 213, 217, 220, 221, 230, 281
 - dual meaning of, 222
- stdarg.h* header file, 378, 395
- stddef.h* header file, 247, 378
- stderr*, 328
- stdin*, 328
- stdin* macro, 115
- stdio.h* header file, 48, 330–32, 378
- stdlib.h* header file, 47, 378, 422
- stdout*, 328
- stepwise refinement, 21, 22–23, 361
- storage class specifiers
 - semantics, 230
- storage classes, 212–35
 - summary, 229–31
- straight line programs, 92
- strcat()* function, 433
- strchr()* function, 434
- strcmp()* function, 434
- strcoll()* function, 434
- strcpy()* function, 189, 435
- strcspn()* function, 435
- streams, 327–29
 - standard, 328
- strength reduction, 209, 241
- strerror()* function, 435
- string constant, 182

- string conversion functions
 - runtime library, 423–26
 - string functions, 195
 - string handling functions, 432–39
 - string literal. *See* string constant
 - string producer macro, 313
 - string.h* header file, 378
 - strings, 181–93
 - arrays of pointers to, 204
 - assigning, 183–85
 - concatenating, 188
 - copying, 189–90
 - declaring, 182–83
 - initializing, 182–83
 - length of, 187–88
 - reading, 186–87
 - standard functions, 195
 - storage, 183, 233
 - vs. chars, 185–86
 - writing, 186–87
 - strlen()* function, 187, 435
 - strncat()* function, 435
 - strncmp()* function, 436
 - strncpy()* function, 436
 - strpbrk()* function, 436
 - strrchr()* function, 437
 - strspn()* function, 437
 - strstr()* function, 190, 437
 - strtod()* function, 424
 - strtok()* function, 437
 - strtol()* function, 425
 - strtoul()* function, 425
 - strtime()* function, 444
 - struct** keyword, 239
 - structure members, 237
 - alignment, 245–47
 - referencing, 240
 - structure template, 237
 - structures, 236–55
 - alignment
 - natural, 246
 - arrays of, 241–42
 - assigning, 255
 - holes in, 245
 - initializing, 239–40
 - name spaces, 244
 - nested, 244
 - passing as function arguments, 252–53
 - returning, 253–55
 - self-referencing, 244–45
 - storage, 238
 - strxfrm()* function, 438
 - stubs, 277
 - subexpressions, 73
 - subroutine, 35
 - subscript, 164
 - subsystems, 14
 - subtraction
 - pointer, 171–72
 - switch** statement, 100
 - syntax, 101–7
 - symbol tables, 365
 - synopses
 - runtime library, 377–79
 - system commands
 - executing
 - system()* function, 429
 - system()* function, 429
 - systems programming languages, 5
- T—
- tabs
 - horizontal
 - escape character sequence `\t`, 68
 - vertical
 - escape character sequence `\v`, 68
 - tag name, 238
 - tan()* function, 389
 - tanh()* function, 389
 - Tannenbaum, A., 15
 - template
 - structure, 237
 - temporary files
 - tmpfile()* function, 421
 - tmpnam()* function, 352
 - tentative definitions, 223

termination signal
 SIGTERM macro, 394
 test engineering, 27
 test suites, 27
 testing, 27–28
 text format, 328
 text stream, 328
 text stream modes, 332
 Thompson, Ken, 5
 tilde
 trigraph sequence, 69
 time functions, 441–46
 time() function, 446
 time.h header file, 378, 441
 time_t type, 442
 tm structure, 442
 TMP_MAX macro, 421
 tmpfile() function, 420
 tmpnam() function, 421
 token pasting, 315
 tokens
 dividing a string into, 437
 tolower() function, 382
 tools
 software, 369–72
 topdown design, 21–23
 toupper() function, 382
 translation limits, 447
 traveling salesman problem, 374
 tree structure, 130
 trees
 binary, 365
 trigonometric functions
 runtime library, 387
 trigraph sequences, 68
 truth tables, 147
 two's complement notation, 61
 type specifiers
 double, 69
 enum, 81–83
 float, 69

long double, 69
 void, 83–84
 typedef name, 239
 typedefs, 84–85
 confusing with **#define**, 83

—U—

UCHAR_MAX macro, 449
 UINT_MAX macro, 449
 Ullman, 368
 ULONG_MAX macro, 449
 unary arithmetic operators, 132–33
 unary minus operator **-**, 132–33
 unary operators, 73
 unary plus operator **+**, 132–33
 unbuffered I/O, 343
 undefined values, 167
 underflow errors, 387
 underscore character **_**, 41
 names beginning with, 462
 ungetc() function, 115, 421
 unions, 262–67
 initializing, 265–66
 name spaces, 244
 storage, 263
 UNIX operating system, 5
 compiling and linking, 34
 unnamed bit fields, 248
 unsigned constants, 67
 unsigned conversions, 76–77
 unsigned data type qualifier, 63
 unsigned integers, 62
 update modes, 400
 user interface, 14
 USHRT_MAX macro, 449

—V—

\v
 vertical tab, 68

- va_arg* macro, 396
- va_end()* function, 396
- va_start* macro, 396
- variable argument lists, 283
 - runtime library, 395–97
- variables, 39–40
 - global, 221–24
 - declaring, 224
- variant records, 236, 266–67
- version skew, 21
- vertical tab
 - escape character sequence `\v`, 68
- vfprintf()* function, 422
- void data type, 57, 83–84, 233
- void type, 275
- volatile specifier, 231
- volatile storage-class modifier, 228–29
- vprintf()* function, 422
- vsprintf()* function, 422

—W—

- wchar_t* type, 423

- wcstombs()* function, 441
- wctomb()* function, 440
- while statement, 107–9
 - syntax, 107
- Wirth, Niklaus, 361
- writing data, 335–40
- writing strings, 186–87

—X—

- X3J11 Technical Committee, 6
- XOR operator `^`, 150
- XOR-assign operator `^=`, 157

—Y—

- yacc* utility, 322

—Z—

- zero
 - representation of, 62